BRIAN GUSTAVO DE OLIVEIRA BRANDÃO

MEGAGAME: SISTEMA DE MARKETPLACE DE JOGOS

Campos do Jordão
2026

BRIAN GUSTAVO DE OLIVEIRA BRANDÃO

MEGAGAME: SISTEMA DE MARKETPLACE DE JOGOS

Projeto Integrador apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), Campus Campos do Jordão, como requisito parcial para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO

Orientador: [NOME DO(A) ORIENTADOR(A)]

Campos do Jordão
2026

BRIAN GUSTAVO DE OLIVEIRA BRANDÃO

MEGAGAME: SISTEMA DE MARKETPLACE DE JOGOS

Projeto Integrador apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), Campus Campos do Jordão, como requisito parcial para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Trabalho aprovado. Campos do Jordão, [DATA DA DEFESA]:

_____

[NOME DO(A) ORIENTADOR(A)]
Orientador(a)

_____

[NOME DO(A) AVALIADOR(A) 1]
Avaliador(a)

_____

[NOME DO(A) AVALIADOR(A) 2]
Avaliador(a)

RESUMO

Este documento apresenta o desenvolvimento de um *marketplace* C2C chamado MegaGame. É voltado para a área de jogos, principalmente os eletrônicos, mas também permitindo a comercialização de diversos itens associados. Tal mercado vem crescendo de forma dramática com o passar dos anos, tanto em retorno financeiro quanto em número de interessados, algo que populariza serviços relacionados ao setor e lhes torna cada vez mais comercialmente viáveis. Com esse contexto em mente, o projeto busca justamente construir um sistema que seja funcional e atrativo para os clientes, sejam eles vendedores, compradores ou ambos simultaneamente.
Palavras-chave: *marketplace*; jogos; gerenciamento; clientes.

ABSTRACT

This document presents the development of a C2C marketplace called MegaGame. Its focus is on the gaming sector, especially electronic games, but also allowing for the commercialization of several related items. The market has been growing dramatically over the years, not only in financial return but also in number of prospective clients, which means related services have become increasingly more popular and commercially viable. With that context in mind, the project aims to build a system that is functional and attractive to clients, whether they're buyers, sellers or both simultaneously.
Keywords: marketplace; games; management; clients.

LISTA DE ABREVIATURAS E SIGLAS

C2C	*Customer-to-customer*
SGBD	Sistema de gerenciamento de banco de dados
SSR	*Server-side rendering*

SUMÁRIO

1 INTRODUÇÃO 9
1.1 Objetivos 9
1.2 Justificativa 9
1.3 Aspectos metodológicos 10
1.4 Aporte teórico 10

2 FUNDAMENTAÇÃO TEÓRICA 11
2.1 Engenharia de software 11
2.1.1 Análise de requisitos 11
2.1.2 Modelagem 12
2.2 Linguagens e ferramentas utilizadas 12
2.2.1 UML 12
2.2.2 Django 14
2.2.3 Supabase 16
2.3 Desenvolvimento do projeto 16

3 PROJETO PROPOSTO 18
3.1 Considerações iniciais 18
3.2 Requisitos 18
3.3 Casos de uso 21
3.3.1 Diagramas de casos de uso 21
3.3.2 Descrições de casos de uso 21
3.4 Arquitetura 30
3.5 Projeto de dados 30
3.5.1 Dicionário de dados 31
3.6 Interfaces 41
3.7 Implementação 48

4 CONCLUSÃO 53

REFERÊNCIAS 54

LISTA DE ILUSTRAÇÕES

Figura 1 – Logo da UML 13
Figura 2 – Logo do Django 14
Figura 3 – Logo do Python 14
Figura 4 – Demonstração do painel administrativo do Django 15
Figura 5 – Logo do Supabase 16
Figura 6 – *Wireframe* da página inicial do MegaGame 17
Figura 7 – Diagrama de casos de uso do gestor 22
Figura 8 – Diagrama de casos de uso do comprador 23
Figura 9 – Diagrama de casos de uso do vendedor 24
Figura 10 – Estrutura de arquivos geral do projeto 31
Figura 11 – Detalhamento da estrutura de arquivos #1 32
Figura 12 – Detalhamento da estrutura de arquivos #2 33
Figura 13 – DER do banco de dados 34
Figura 14 – Captura de tela da *home* do MegaGame 41
Figura 15 – Captura de tela da pesquisa do MegaGame 41
Figura 16 – Captura de tela das categorias do MegaGame 42
Figura 17 – Captura de tela da busca por categoria do MegaGame 42
Figura 18 – Captura de tela da edição de perfil do MegaGame 43
Figura 19 – *Dashboard* de gestor do MegaGame 43
Figura 20 – Captura de tela do *login* do MegaGame 44
Figura 21 – Captura de tela do cadastro do MegaGame 44
Figura 22 – Captura de tela da lista de anúncios do MegaGame 45
Figura 23 – Captura de tela do *dropdown menu* do MegaGame 45
Figura 24 – Captura de tela de um *pop-up* de aviso do MegaGame 46
Figura 25 – Captura de tela da edição de anúncio do MegaGame 46
Figura 26 – Captura de tela da criação de anúncio do MegaGame 47
Figura 27 – Criação de variações do MegaGame 47
Figura 28 – Rascunhos de anúncio do MegaGame 48
Figura 29 – *Dashboard* de vendedor do MegaGame 49
Figura 30 – Captura de tela dos pedidos recebidos do MegaGame 49
Figura 31 – Captura de tela dos anúncios do MegaGame 50
Figura 32 – Captura de tela do carrinho do MegaGame 50
Figura 33 – Captura de tela dos pedidos feitos do MegaGame 51
Figura 34 – Avaliação de produto do MegaGame 51
Figura 35 – Avaliação de vendedor do MegaGame 52

LISTA DE QUADROS

Quadro 1 – Requisitos funcionais 18
Quadro 2 – Requisitos não funcionais 20
Quadro 3 – Cadastrar conta 21
Quadro 4 – Efetuar login 21
Quadro 5 – Navegar pelas categorias 23
Quadro 6 – Pesquisar produto 24
Quadro 7 – Editar perfil 25
Quadro 8 – Analisar o *dashboard* administrativo 25
Quadro 9 – Alterar taxa de comissão 25
Quadro 10 – Adicionar produto ao carrinho 26
Quadro 11 – Fazer pedido 26
Quadro 12 – Receber produto 26
Quadro 13 – Avaliar vendedor 27
Quadro 14 – Avaliar produto 27
Quadro 15 – Cancelar pedido 27
Quadro 16 – Analisar *dashboard* de vendas 28
Quadro 17 – Ver pedidos recebidos 28
Quadro 18 – Preparar entrega 28
Quadro 19 – Publicar anúncio 29
Quadro 20 – Editar anúncio 29
Quadro 21 – Retirar anúncio do ar 29
Quadro 22 – Deletar anúncio 30
Quadro 23 – Usuários 34
Quadro 24 – Categorias 35
Quadro 25 – Perfis de usuário 35
Quadro 26 – Carrinhos 35
Quadro 27 – Produtos 36
Quadro 28 – Avaliações de vendedor 36
Quadro 29 – Avaliações de produto 37
Quadro 30 – Imagens de produto 37
Quadro 31 – Variações de produto 38
Quadro 32 – Pedidos 38
Quadro 33 – Estoque 39
Quadro 34 – Itens do carrinho 39
Quadro 35 – Configurações da plataforma 40
Quadro 36 – Comissões 40

1 INTRODUÇÃO

A indústria de jogos eletrônicos atualmente forma um dos maiores mercados da área de entretenimento, movimentando centenas de bilhões de dólares ao todo. Segundo pesquisas como as da Newzoo (s.d.), Brasil costuma estar entre os dez países que mais contribuem para tal situação, possuindo uma forte comunidade *gamer*.

Sendo assim, este projeto tem como objetivo apresentar um *marketplace* C2C chamado MegaGame, voltado para essa audiência. Além da oportunidade comercial, o desenvolvimento do site também foi motivado pela observação do autor de que seria útil existir um local centralizado para o comércio de itens relacionados ao tema, tendo em vista a grande dispersão através de diversas plataformas (OLX, Mercado Livre, Shopee, dentre outras) em decorrência da falta de um serviço dedicado ao segmento.

Para o desenvolvimento do MegaGame, foram utilizados a linguagem Python (com o *framework* Django) e o SGBD PostgreSQL (através da plataforma Supabase).

1.1 Objetivos

O objetivo da plataforma criada neste trabalho é fornecer um serviço eficiente, intuitivo, estável e responsivo, utilizando tecnologias difundidas no ramo da tecnologia para viabilizar esses requisitos. Já o objetivo deste documento em específico é descrever toda a criação do MegaGame, detalhando o processo de desenvolvimento e fornecendo informações sobre suas funcionalidades e casos de uso.

1.2 Justificativa

Conforme citado anteriormente, a justificativa deste trabalho se dá pela observação do autor de que há uma lacuna (ao menos no cenário brasileiro) na indústria de *games* que pode ser preenchida por uma plataforma com as características do MegaGame, além da crescente viabilidade comercial do setor.

Logo, o sistema se torna ideal tanto para os clientes quanto para seu(s) gestor(es), demonstrando assim sua relevância para todos os *stakeholders* envolvidos, conforme ressaltam os conteúdos referentes à gestão de projetos abordados durante o curso.

1.3 Aspectos metodológicos

As ferramentas utilizadas durante a criação do projeto foram a linguagem de programação Python no *back end*, através do *framework* Django e utilizando a prática de SSR; a linguagem de programação JavaScript no *front end*, aliada às linguagens de *markdown* e *style sheets* HTML e CSS (respectivamente); e além disso, o SGBD PostgreSQL, através da plataforma online Supabase para hospedagem na nuvem.

Já a ferramenta utilizada para criação dos diagramas da linguagem de modelagem UML exibidos no documento foi o draw.io, mais especificamente sua versão *web* (também chamada de diagrams.net).

1.4 Aporte teórico

Para os estudos teóricos acerca do tema abordado, foram consultados alguns dos livros mais populares e acessíveis sobre tópicos como o UML, a engenharia de *software* e o gerenciamento de bancos de dados, além de fontes virtuais que comprovem alguns dos conceitos que embasam o projeto como um todo.

Mais especificamente, os escritos de autores como Coronel, Pressman e etc. foram as responsáveis por ajudar a definir os princípios guiadores do trabalho, graças aos métodos, práticas e paradigmas estabelecidos e discutidos por tais obras.

2 FUNDAMENTAÇÃO TEÓRICA

Nesta seção, serão apresentadas em maiores detalhes as tecnologias e teorias empregadas durante a construção do projeto.

2.1 Engenharia de software

O conceito de engenharia de *software* é utilizado para se referir a uma área da computação voltada para a aplicação de certos princípios e ferramentas, com o intuito de esquematizar, desenvolver e gerenciar sistemas. Seu propósito é garantir que todos os requisitos determinados no início de um projeto sejam cumpridos de forma eficaz, mas também de forma eficiente quando possível; isto é, assegurando que tudo esteja dentro do prazo estabelecido, respeitando os limites orçamentários e se encaixando em padrões de qualidade do mercado (como as normas ISO, por exemplo).

Atualmente, no contexto acadêmico, são duas as principais referências neste setor – as obras de Pressman e Maxim, junto das obras de Sommerville. Pressman e Maxim (2021, p. 59) dizem, acerca do tema:

> O Instituto de Engenheiros Eletricistas e Eletrônicos (IEEE) elaborou a seguinte definição para engenharia de *software*: [...] a aplicação de uma abordagem sistemática, disciplinada e quantificável no desenvolvimento, na operação e na manutenção de *software*; isto é, a aplicação de engenharia ao *software*.

Já Sommerville (2011, p. 22) afirma:

> A engenharia de *software* tem por objetivo apoiar o desenvolvimento profissional de *software*, mais do que a programação individual. Ela inclui técnicas que apoiam especificação, projeto e evolução de programas, que normalmente não são relevantes para o desenvolvimento de *software* pessoal.

As etapas da engenharia de *software* são organizadas em processos, para os quais são levantados os requisitos correspondentes. Assim, os melhores métodos possíveis podem ser empregados para cumpri-los.

2.1.1 Análise de requisitos

Existem três tipos diferentes de requisito, sendo dois deles os principais (mais precisamente, os funcionais e os não-funcionais). Primeiramente, os requisitos funcionais descrevem as funcionalidades exatas que um sistema deve possuir. Por exemplo,

um site que trabalhe com perfis de usuário deve permitir que seus clientes cadastrem contas para si e depois efetuem o *login* nelas.

Já os requisitos não-funcionais referem-se aos atributos de qualidade que o sistema deve alcançar. Por instância, suportar X usuários simultâneos, carregar todo o conteúdo em até Y segundos, garantir conformidade com leis de proteção de dados de determinado território, dentre outros casos.

E por fim, existem também os requisitos de domínio. Tecnicamente, eles irão cair dentro de uma das categorias anteriores, mas vale a pena ressaltá-los a parte pois correspondem às regras de negócio de um determinado setor; isto é, são formados a partir da análise daquela área em específico, para garantir que suas particularidades estejam sendo observadas e tratadas.

Para obter todos os requisitos, é realizada uma ação conhecida como levantamento, que pode englobar várias táticas. Análise de mercado, entrevistas com peritos, discussões de ideias com o cliente e/ou outros desenvolvedores (processo também conhecido como *brainstorming*) e demais práticas são passíveis de serem utilizadas durante essa etapa do planejamento.

2.1.2 Modelagem

A partir do momento em que os requisitos foram todos levantados e já existe uma ideia do rumo que o projeto deve tomar, é uma boa prática fazer a modelagem do *software*, através de ferramentas como o UML (cujo uso será mais detalhado posteriormente, por se tratar de uma linguagem utilizada durante a construção deste trabalho em específico).

Modelagem, neste cenário, significa representar o sistema sendo desenvolvido através de recursos visuais, facilitando sua compreensão, análise e posterior implementação. É especialmente útil para trabalhos em equipe ou que envolvem *stakeholders* externos, mas também pode se mostrar de suma importância mesmo em criações individuais.

Ela pode abranger diversas áreas. No ecossistema da UML, por exemplo, existem diagramas de casos de uso, diagramas de classe (que representam a estrutura do sistema, através do conceito da programação orientada a objetos), diagramas de sequência (para detalhar as atividades que o sistema realiza), dentre outros.

2.2 Linguagens e ferramentas utilizadas

2.2.1 UML

UML significa Unified Modeling Language (“linguagem unificada de modelagem”, em inglês), que é o nome de uma linguagem de modelagem desenvolvida na década

de 1990 por Grady Booch, James Rumbaugh e Ivar Jacobson, a partir da unificação de suas três linguagens desenvolvidas de forma independente em um “pacote” coeso.



Figura 1 – Logo da UML

Fonte: Wikipedia (2026c).

Desde 1997, a UML é gerida e padronizada pelo Object Management Group (ou OMG). Possui diversos usuários no meio de desenvolvimento de *software*, inclusive Gilleanes Guedes, cujas obras servem como um guia para sua utilização ideal. Guedes (2018, p. 24) afirma:

> Um modelo de software captura uma visão de um sistema físico; é uma abstração do sistema com um certo propósito, como descrever aspectos estruturais ou comportamentais do *software*. Esse propósito determina o que deve ser incluído no modelo e o que é considerado irrelevante. Assim, um modelo descreve completamente aqueles aspectos do sistema físico relevantes ao propósito do modelo, no nível apropriado de detalhe.

Os diagramas da UML são divididos em dois tipos: os estruturais e os comportamentais. Os primeiros focam na organização estática do sistema (como os diagramas de classes, objetos, componentes, implantação...) e os últimos focam na dinâmica e interações do sistema (como os diagramas de casos de uso, sequência, estados...)

2.2.2 Django

O Django é um *framework* para a linguagem Python, desenvolvido em 2005 por Adrian Holovaty e Simon Willison, sendo mantido desde então pela Django Software Foundation. O Python, por sua vez, fora desenvolvido em 1991 pelo neerlandês Guido van Rossum para suceder uma linguagem anterior conhecida como ABC, e é uma das linguagens de programação mais populares e acessíveis do mundo.



Figura 2 – Logo do Django

Fonte: Wikipedia (2026a).



Figura 3 – Logo do Python

Fonte: Wikipedia (2026b).

Em comparação com outros *frameworks* para Python, como o Flask, o Django se destaca por ser voltado ao desenvolvimento *web*. É utilizado por diversas empresas, incluindo o próprio IFSP, em sua plataforma virtual SUAP. Trabalha com o conceito de componentes (similar a alguns outros *frameworks web* populares, como Angular, este em especíico sendo visto no decorrer do curso), permitindo alta modularização, e providencia uma interface administrativa embutida para facilitar a gestão.

Através da prática do SSR, é possível utilizá-lo para exibir o site sem necessidade de *frameworks* adicionais de *front end*. Sendo assim, se tratando da questão *web*, a combinação usual de HTML/CSS/JavaScript já foi suficiente pra atingir todas as metas almejadas.



Figura 4 – Demonstração do painel administrativo do Django

Fonte: Wikipedia (2026a).

2.2.3 Supabase

O Supabase é uma plataforma que permite o armazenamento de um banco de dados na nuvem. Trabalha especificamente com PostgreSQL, mas é possível fazer a conversão automática do código incluso no repositório do projeto em trechos correspondentes de SQL que criem as tabelas e campos necessários no banco, através do processo de *migration*.

Foi criada em 2020 na Singapura por Paul Copplestone e Ant Wilson, como uma alternativa *open-source* ao Firebase, serviço com uma proposta similar que é de posse do Google.



Figura 5 – Logo do Supabase

Fonte: Supabase (2026).

Vale notar que, em testes locais, o SQLite foi utilizado por sua performance, visto que em tais circunstâncias não era necessário armazenar o conteúdo gerado a longo prazo.

2.3 Desenvolvimento do projeto

O projeto começou com a ideia inicial, que surgiu a partir dos gostos pessoais do autor deste trabalho, com o auxílio das sugestões do orientador da disciplina. A partir dela, foi feita uma pesquisa de mercado para determinar as regras de negócio do setor, além da confecção de *wireframes* pras principais páginas do site, de modo a definir o design geral.

Daí veio o código, sendo que todos os aspectos (*front*, *back* e banco de dados) foram trabalhados simultaneamente. Também foram feitos testes para tratar exceções e garantir que as funcionalidades estivessem alinhadas com o escopo do projeto e protegidas contra brechas de segurança.

Como último passo, veio a formalização da documentação e da modelagem, conforme demonstrado ao decorrer deste texto.

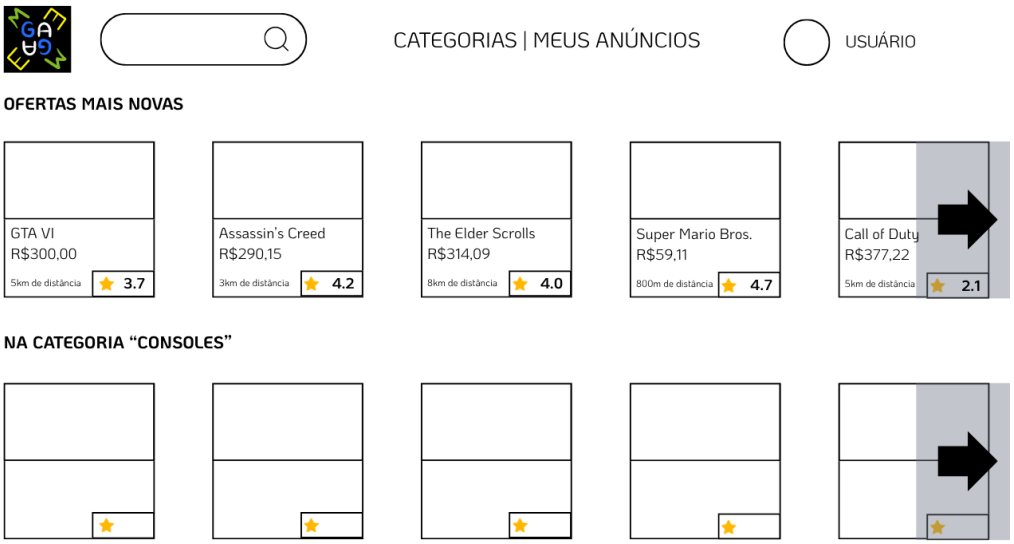


Figura 6 – *Wireframe* da página inicial do MegaGame

Fonte: Autor (2026).

3 PROJETO PROPOSTO

Nesta seção, serão apresentados os detalhes por trás do funcionamento do MegaGame, além de seus modelos, instruções de uso e *proof of concept* (prova de conceito) visual.

3.1 Considerações iniciais

Reiterando o que foi supracitado em seções anteriores, o sistema é feito em Django (com uso de SSR), consequentemente utilizando arquitetura baseada em componentes, trabalhando assim os conceitos da programação modular. Para o *front*, foi utilizado HTML/CSS/JavaScript conforme o padrão do desenvolvimento *web*. E pro banco de dados, foi empregado o PostgreSQL através da plataforma Supabase para garantir armazenamento em nuvem.

3.2 Requisitos

Os requisitos funcionais do projeto foram:

Quadro 1 – Requisitos funcionais

Código	Requisito	Prioridade
RF1	O sistema deve permitir que o usuário efetue *login*.	Alta
RF2	O sistema deve permitir que novos usuários se cadastrem.	Alta
RF3	O sistema deve permitir a alteração dos dados de perfil após criação da conta de usuário.	Alta
RF4	O sistema deve manter um histórico completo de todas as ações realizadas, seja pelo(s) gestor(es) ou pelos clientes.	Alta
RF5	O sistema deve possuir *dashboards*, tanto para o(s) gestor(es) quanto para os clientes, para que eles possam fazer o rastreio da parte financeira.	Alta
RF6	O sistema deve permitir que o usuário busque anúncios específicos através de uma barra de pesquisa.	Média

Código	Requisito	Prioridade
RF7	O sistema deve permitir a compra e venda de produtos através de anúncios.	Alta
RF8	O sistema deve permitir a edição e exclusão de anúncios depois que eles forem ao ar.	Alta
RF9	O sistema deve permitir a categorização de produtos.	Média
RF10	O sistema deve permitir a criação de variações para um produto, cada qual com suas informações específicas (nome, preço, estoque...)	Média
RF11	O sistema deve estar de acordo com as práticas de responsividade e acessibilidade vigentes no mercado.	Baixa
RF12	O sistema deve permitir que um usuário rastreie todo o fluxo de envio do produto que comprou antes de tê-lo em mãos.	Alta
RF13	O sistema deve permitir que um usuário tenha o direito de cancelar seu pedido em determinados momentos, incluindo em um certo espaço de tempo após a entrega.	Alta
RF14	O sistema deve permitir que seus usuários façam avaliações, tanto dos vendedores dos quais compraram quanto dos produtos que obtiveram.	Baixa
RF15	O sistema deve permitir que seus usuários naveguem pelas diferentes categorias cadastradas para ver todos os anúncios pertencentes a determinada categoria.	Baixa
RF16	O sistema deve permitir que o gestor altere a taxa de comissão recebida sobre as vendas.	Alta
RF17	O sistema deve permitir que o comprador possa colocar produtos em um carrinho, para então finalizar o pedido quando estiver satisfeito com suas escolhas.	Alta
RF18	O sistema deve permitir que uma criação de anúncio interrompida possa ser retomada posteriormente.	Média

Fonte: Autor (2026).

Já os não-funcionais foram:

Quadro 2 – Requisitos não funcionais

Código	Requisito	Prioridade
RNF1	O sistema deve carregar todo o conteúdo de uma determinada página em até 5 segundos.	Média
RNF2	O sistema deve suportar até 1000 usuários simultâneos sem que haja perda de desempenho.	Alta
RNF3	O sistema deve ser compatível com Windows 10 ou superior.	Alta
RNF4	O sistema deve garantir disponibilidade mínima de 99% ao longo de um ano.	Alta
RNF5	O sistema deve estar de acordo com a LGPD (Lei Geral de Proteção de Dados).	Alta
RNF6	O sistema deve ser projetado para facilitar manutenção e eventuais atualizações, fazendo com que haja tempo mínimo de indisponibilidade em tais casos.	Média
RNF7	O sistema deve possuir documentação técnica completa.	Baixa
RNF8	A interface do sistema deve ser intuitiva, permitindo aprendizado suficiente para conduzir atividades num máximo de 1 hora.	Alta
RNF9	O sistema deve validar informações-chave do usuário durante um cadastro ou edição, como o email ou a senha.	Alta
RNF10	O sistema deve tratar eventuais exceções para evitar brechas de segurança que possam causar prejuízo.	Alta
RNF11	O sistema deve possuir uma barra de navegação (nav).	Média
RNF12	O sistema deve poder ser acessado pelos usuários apenas através de uma sessão ativa em suas respectivas contas.	Alta
RNF13	O sistema deve ser construído com uma infraestrutura robusta, escalável, segura e estável.	Alta

Fonte: Autor (2026).

3.3 Casos de uso

Nesta subseção, serão apresentados os diagramas gerais de casos de uso e os casos de uso individuais em forma de documentação, para maiores detalhes.

A definição de casos de uso envolve uma sequência de interações entre um "ator" do sistema (seja este um usuário ou até mesmo outro sistema) e o sistema em si, gerando assim um resultado de valor para o ator em questão, conforme corroborado por Pressman e Maxim (2021). Eles ajudam a definir o comportamento desejado de um sistema em determinados cenários.

3.3.1 Diagramas de casos de uso

Os diagramas de casos de uso mostrados contemplam o(s) gestor(es), os usuários compradores e os usuários vendedores.

3.3.2 Descrições de casos de uso

Estes são os casos de uso mostrados na subseção anterior, porém sob uma perspectiva individual e detalhada.

Quadro 3 – Cadastrar conta

Nome do Caso de Uso	Cadastrar Conta
Ator Principal	Vendedor; Comprador
Ator Secundário	
Resumo	Cadastrar uma nova conta de usuário no sistema.
Pré-Condições	
Pós-Condições	Efetuar Login

Fonte: Autor (2026).

Quadro 4 – Efetuar login

Nome do Caso de Uso	Efetuar Login
Ator Principal	Comprador; Vendedor
Ator Secundário	
Resumo	Efetuar login em uma conta de usuário cadastrada no sistema.
Pré-Condições	Cadastrar Conta
Pós-Condições	

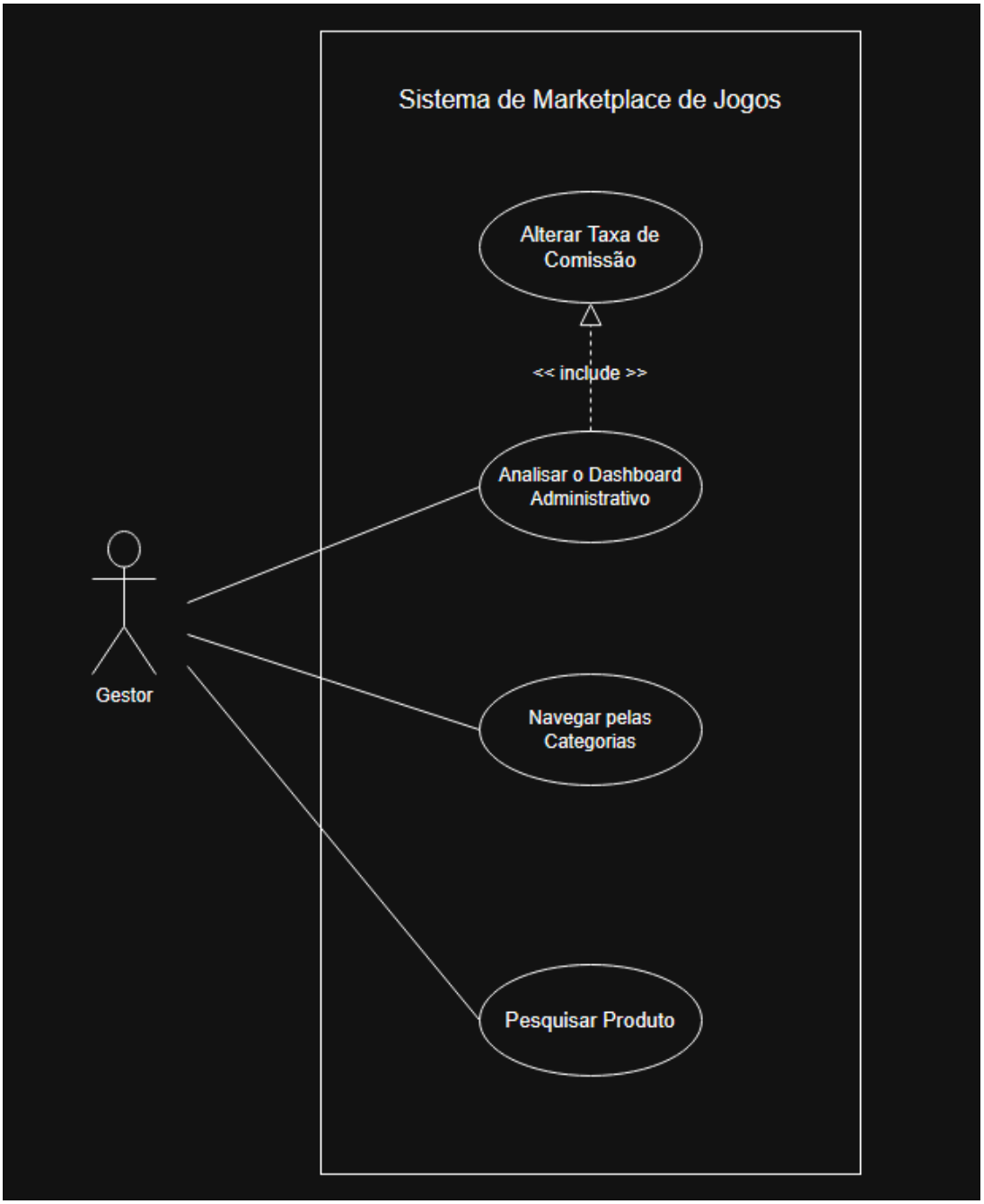


Figura 7 – Diagrama de casos de uso do gestor

Fonte: Autor (2026).

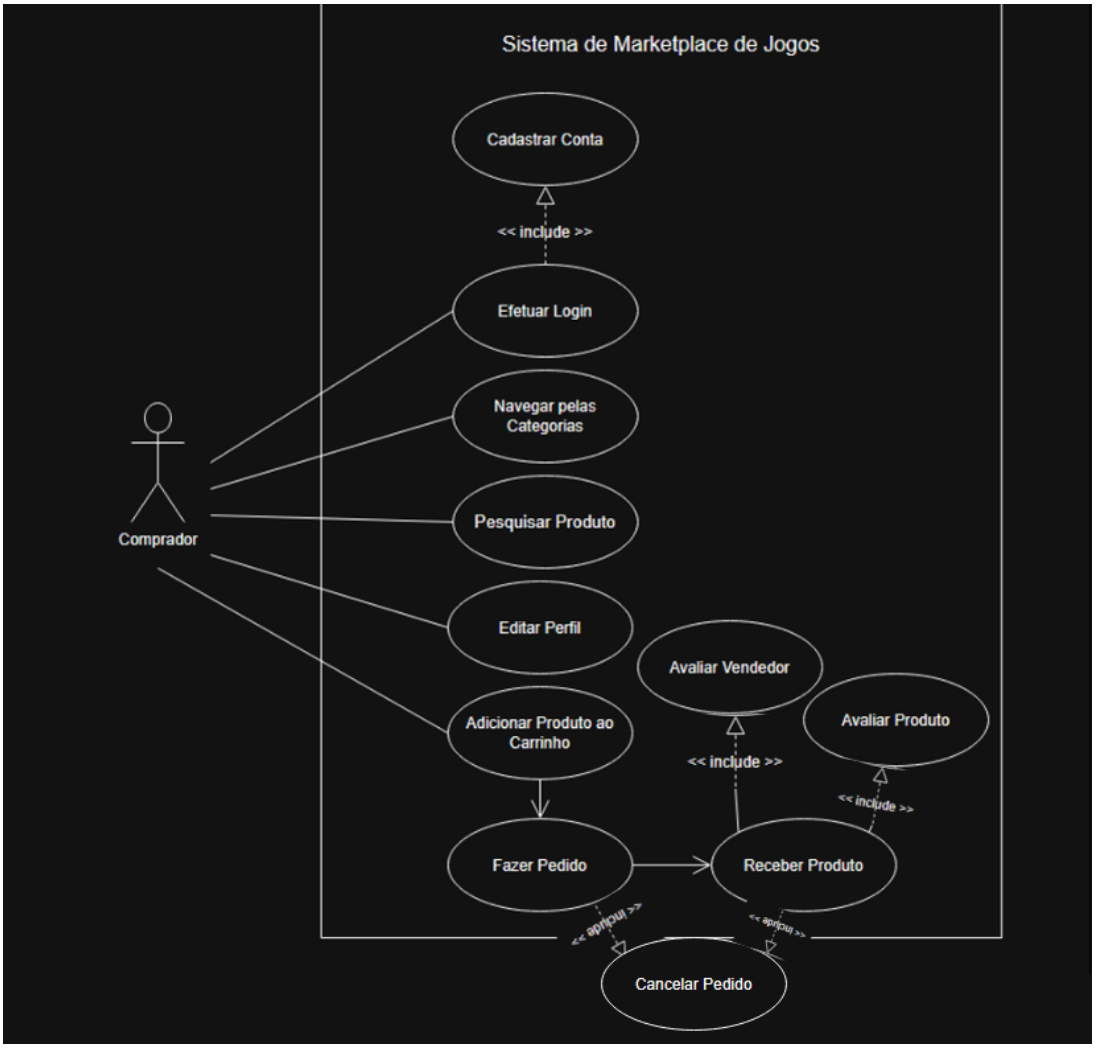


Figura 8 – Diagrama de casos de uso do comprador

Fonte: Autor (2026).

Fonte: Autor (2026).

Quadro 5 – Navegar pelas categorias

Nome do Caso de Uso	Navegar pelas Categorias
Ator Principal	Gestor; Comprador; Vendedor
Ator Secundário	
Resumo	Navegar pela aba de “Categorias” e ver os diferentes produtos de cada categoria.
Pré-Condições	
Pós-Condições	

Fonte: Autor (2026).

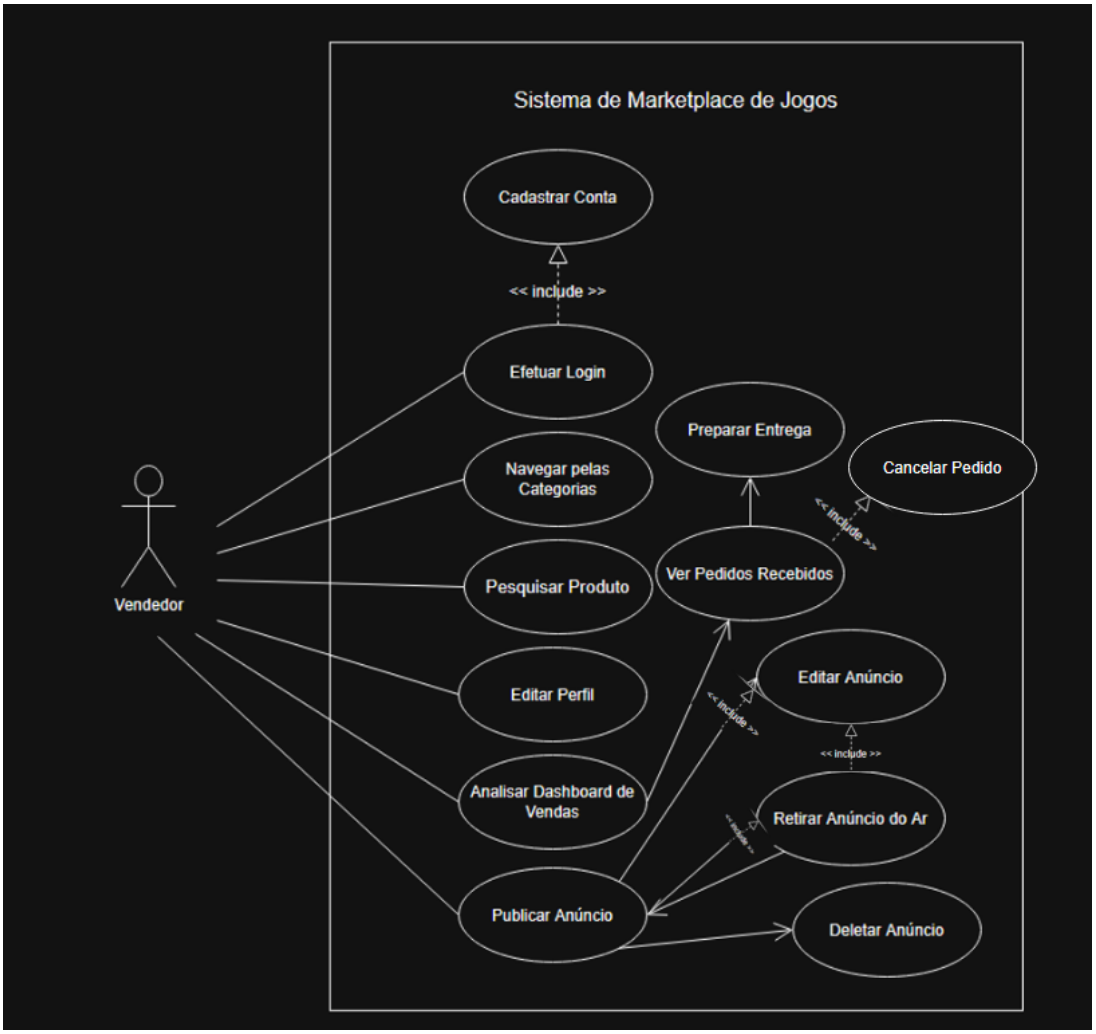


Figura 9 – Diagrama de casos de uso do vendedor

Fonte: Autor (2026).

Quadro 6 – Pesquisar produto

Nome do Caso de Uso	Pesquisar Produto
Ator Principal	Gestor; Vendedor; Comprador
Ator Secundário	
Resumo	Pesquisar um produto na barra de busca do sistema.
Pré-Condições	
Pós-Condições	

Fonte: Autor (2026).

Quadro 7 – Editar perfil

Nome do Caso de Uso	Editar Perfil
Ator Principal	Vendedor; Comprador
Ator Secundário	
Resumo	Editar as informações inseridas no perfil (nome, senha, localização...)
Pré-Condições	
Pós-Condições	

Fonte: Autor (2026).

Quadro 8 – Analisar o *dashboard* administrativo

Nome do Caso de Uso	Analisar o Dashboard Administrativo
Ator Principal	Gestor
Ator Secundário	
Resumo	Observar o painel administrativo para ver os relatórios do sistema e estatísticas de venda.
Pré-Condições	
Pós-Condições	Alterar a Taxa de Comissão

Fonte: Autor (2026).

Quadro 9 – Alterar taxa de comissão

Nome do Caso de Uso	Alterar Taxa de Comissão
Ator Principal	Gestor
Ator Secundário	
Resumo	Alterar a taxa da comissão obtida através das transações.
Pré-Condições	Analisar o Dashboard Administrativo
Pós-Condições	

Fonte: Autor (2026).

Quadro 10 – Adicionar produto ao carrinho

Nome do Caso de Uso	Adicionar Produto ao Carrinho
Ator Principal	Comprador
Ator Secundário	
Resumo	O comprador adiciona um produto que gostaria de comprar ao seu carrinho.
Pré-Condições	
Pós-Condições	Fazer Pedido

Fonte: Autor (2026).

Quadro 11 – Fazer pedido

Nome do Caso de Uso	Fazer Pedido
Ator Principal	Comprador
Ator Secundário	
Resumo	O usuário finaliza o pedido com todos os itens presentes em seu carrinho.
Pré-Condições	Adicionar Produto ao Carrinho
Pós-Condições	Cancelar Pedido; Receber Produto

Fonte: Autor (2026).

Quadro 12 – Receber produto

Nome do Caso de Uso	Receber Produto
Ator Principal	Comprador
Ator Secundário	Vendedor
Resumo	Após a preparação da entrega, o comprador recebe o produto.
Pré-Condições	Fazer Pedido
Pós-Condições	Cancelar Pedido; Avaliar Vendedor; Avaliar Produto

Fonte: Autor (2026).

Quadro 13 – Avaliar vendedor

Nome do Caso de Uso	Avaliar Vendedor
Ator Principal	Comprador
Ator Secundário	
Resumo	Após receber um produto que comprou, o usuário avalia o vendedor.
Pré-Condições	Receber Produto
Pós-Condições	

Fonte: Autor (2026).

Quadro 14 – Avaliar produto

Nome do Caso de Uso	Avaliar Produto
Ator Principal	Comprador
Ator Secundário	
Resumo	O usuário avalia um produto que comprou após recebê-lo.
Pré-Condições	Receber Produto
Pós-Condições	

Fonte: Autor (2026).

Quadro 15 – Cancelar pedido

Nome do Caso de Uso	Cancelar Pedido
Ator Principal	Comprador; Vendedor
Ator Secundário	
Resumo	O pedido é cancelado, seja pelo comprador ou pelo vendedor. Isso pode inclusive ser feito após a entrega do produto, por um determinado período.
Pré-Condições	Fazer Pedido; Receber Produto; Ver Produtos Recebidos
Pós-Condições	

Fonte: Autor (2026).

Quadro 16 – Analisar *dashboard* de vendas

Nome do Caso de Uso	Analisar Dashboard de Vendas
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor analisa seu dashboard de vendas para ver suas estatísticas e relatórios.
Pré-Condições	
Pós-Condições	Ver Pedidos Recebidos

Fonte: Autor (2026).

Quadro 17 – Ver pedidos recebidos

Nome do Caso de Uso	Ver Pedidos Recebidos
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor vê os pedidos que recebeu de compradores.
Pré-Condições	Analisar Dashboard de Vendas
Pós-Condições	Cancelar Pedido; Preparar Entrega

Fonte: Autor (2026).

Quadro 18 – Preparar entrega

Nome do Caso de Uso	Preparar Entrega
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor prepara a entrega de um produto para o comprador.
Pré-Condições	Ver Pedidos Recebidos
Pós-Condições	

Fonte: Autor (2026).

Quadro 19 – Publicar anúncio

Nome do Caso de Uso	Publicar Anúncio
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor publica o anúncio de um produto para vendê-lo.
Pré-Condições	Retirar Anúncio do Ar
Pós-Condições	Retirar Anúncio do Ar; Editar Anúncio

Fonte: Autor (2026).

Quadro 20 – Editar anúncio

Nome do Caso de Uso	Editar Anúncio
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor edita as informações de um anúncio após sua publicação inicial.
Pré-Condições	Publicar Anúncio
Pós-Condições	

Fonte: Autor (2026).

Quadro 21 – Retirar anúncio do ar

Nome do Caso de Uso	Retirar Anúncio do Ar
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor retira o anúncio do ar temporariamente. Ele pode ainda editá-lo nesse estado.
Pré-Condições	Publicar Anúncio
Pós-Condições	Publicar Anúncio; Editar Anúncio

Fonte: Autor (2026).

Quadro 22 – Deletar anúncio

Nome do Caso de Uso	Deletar Anúncio
Ator Principal	Vendedor
Ator Secundário	
Resumo	O vendedor deleta um anúncio (ao menos para si e outros usuários comuns; os registros ainda ficarão no sistema por garantia).
Pré-Condições	Publicar Anúncio
Pós-Condições	

Fonte: Autor (2026).

3.4 Arquitetura

A arquitetura do sistema é dividida em diversos componentes, como é padrão do Django. As imagens mostram as pastas específicas em maiores detalhes, sendo que todos os componentes possuem os mesmos arquivos-base, exceto por core (o "núcleo", que é padrão na criação de um novo projeto, ao invés de ser criado pelo desenvolvedor).

3.5 Projeto de dados

A principal fonte acadêmica a respeito do tema "banco de dados" é Coronel e suas obras, com vários coautores ao decorrer dos anos, tais como Morris e Rob. Morris e Coronel (2016, p. 37) afirmam, acerca do design de dados que forma a base de um projeto:

> Design de banco de dados se refere às atividades que focam no design da estrutura do banco de dados que será utilizada para armazenar e gerenciar dados de usuários do sistema. Um banco de dados que cumpre todos os requisitos de usuário não surge por mágica; sua estrutura deve ser planejada cuidadosamente. [...] Até um SGBD bom terá uma performance ruim com um banco de dados de design ruim.

Sendo assim, o Supabase foi escolhido para se alinhar com esses princípios. Através de sua integração com o Django para viabilizar o processo de *migration*, e

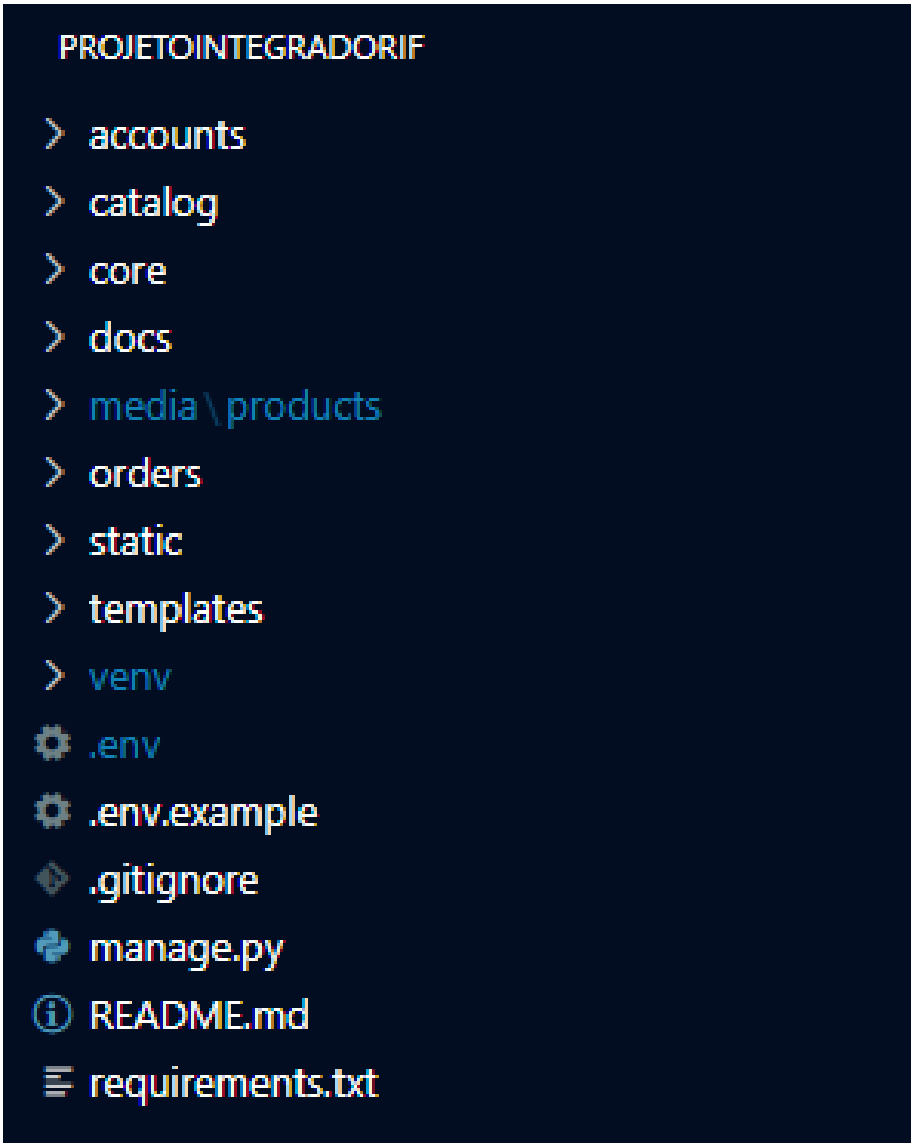


Figura 10 – Estrutura de arquivos geral do projeto

Fonte: Autor (2026).

de sua interface *web* intuitiva, ele permite conveniência e rapidez na gestão do design de banco de dados, sendo que ele automaticamente cria um diagrama entidade-relacionamento (DER) de todo o banco e também possui um editor SQL embutido caso precisem ser feitas mudanças espontâneas.

3.5.1 Dicionário de dados

Aqui, serão apresentados os dicionários de dados das tabelas relevantes ao projeto (tendo em mente que muitas das tabelas vistas no DER acima são componentes padrão do Django).

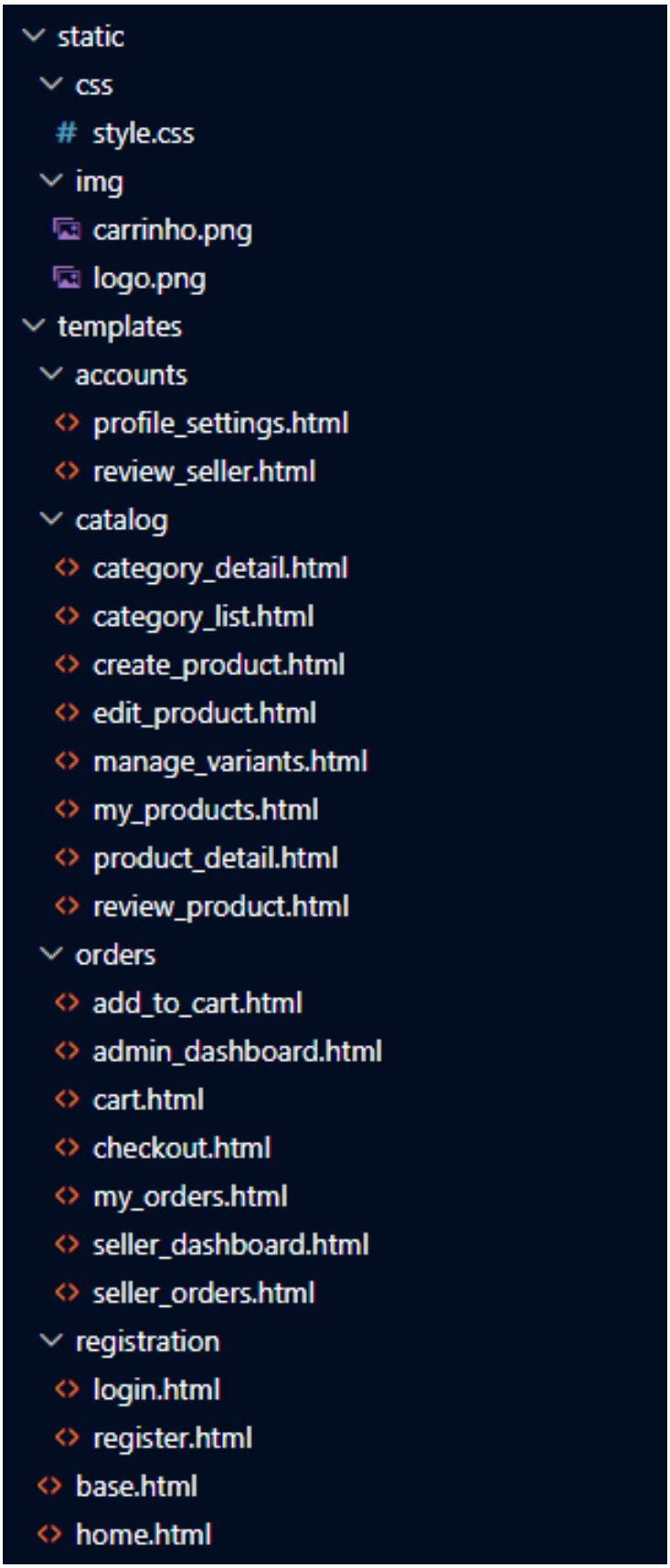


Figura 11 – Detalhamento da estrutura de arquivos #1

Fonte: Autor (2026).

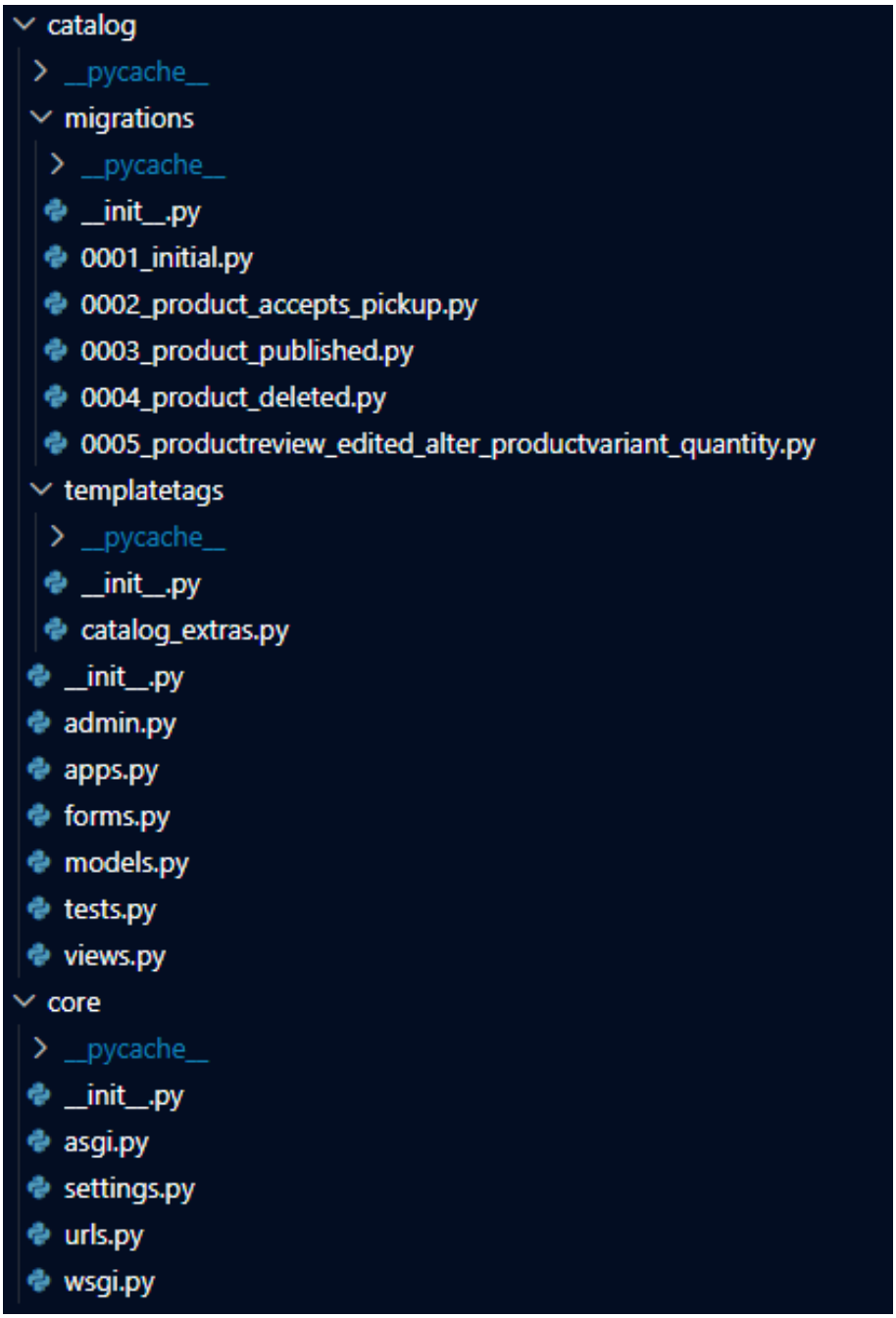


Figura 12 – Detalhamento da estrutura de arquivos #2

Fonte: Autor (2026).

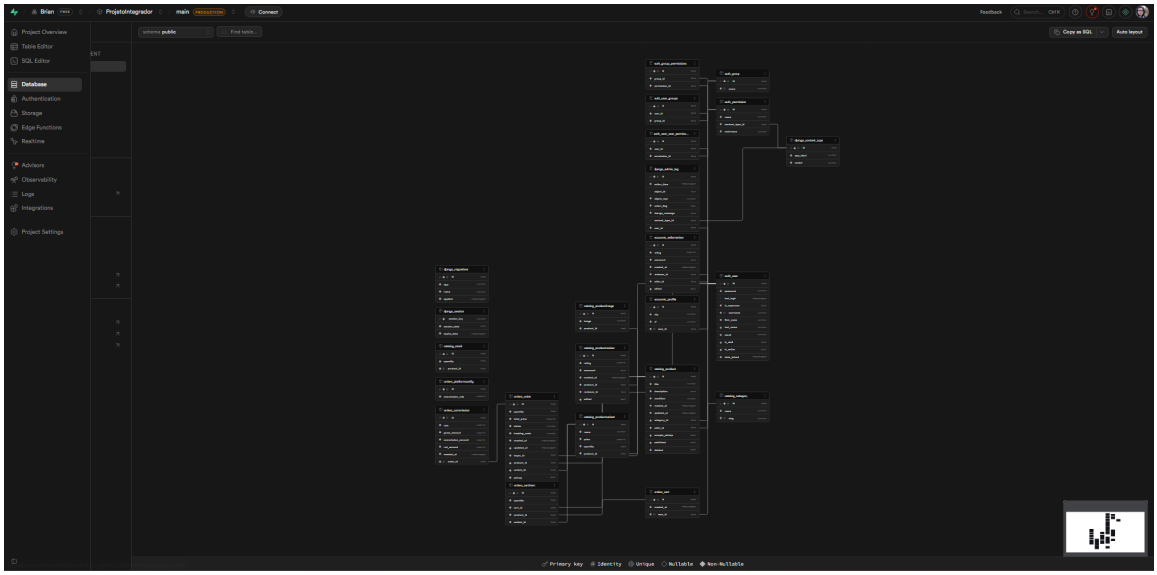


Figura 13 – DER do banco de dados

Fonte: Autor (2026).

Quadro 23 – Usuários

Tabela de Usuários

Nome	Conteúdo	Tipo	Obrigatoriedade	Chave?
id	ID do usuário	int4	SIM	PK
password	Senha do usuário	varchar	SIM	
last_login	Data do último login do usuário	timestamptz	SIM	
is_superuser	O usuário é um superusuário do sistema?	bool	SIM	
username	Nome do usuário dentro da plataforma	varchar	SIM	
first_name	Nome real do usuário	varchar	SIM	
last_name	Sobrenome do usuário	varchar	SIM	
email	Email do usuário	varchar	SIM	
is_staff	O usuário faz parte da equipe por trás do sistema?	bool	SIM	
is_active	O usuário é ativo?	bool	SIM	
date_joined	Data na qual o usuário criou sua conta	timestamptz	SIM	

Fonte: Autor (2026).

Quadro 24 – Categorias

Tabela de Categorias				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID da categoria	int8	SIM	PK
name	Nome da categoria	varchar	SIM	
slug	*Slug* da categoria (pra gerar a URL)	varchar	SIM	

Fonte: Autor (2026).

Quadro 25 – Perfis de usuário

Tabela de Perfis de Usuário				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID do perfil	int8	SIM	PK
city	Cidade do usuário	varchar	SIM	
uf	UF (unidade federativa) do usuário	varchar	SIM	
user_id	ID do usuário	int4	SIM	FK

Fonte: Autor (2026).

Quadro 26 – Carrinhos

Tabela de Carrinhos				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID do carrinho	int8	SIM	PK
created_at	Data de criação do carrinho	timestamptz	SIM	
user_id	ID do usuário ao qual o carrinho pertence	int4	SIM	FK

Fonte: Autor (2026).

Quadro 27 – Produtos

Tabela de Produtos				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID do produto	int8	SIM	PK
title	Nome do produto	varchar	SIM	
description	Descrição do produto	text	SIM	
condition	Condição de uso do produto	varchar	SIM	
created_at	Data de criação do produto	timestamptz	SIM	
updated_at	Data de atualização do produto (se houve)	timestamptz	NÃO	
category_id	ID da categoria a qual o produto pertence	int8	SIM	FK
seller_id	ID do usuário vendendo o produto	int4	SIM	FK
accepts_ pickup	O produto aceita retirada em mãos?	bool	SIM	
published	O produto está no ar?	bool	SIM	
deleted	O produto foi deletado pelo usuário?	bool	SIM	

Fonte: Autor (2026).

Quadro 28 – Avaliações de vendedor

Tabela de Avaliações de Vendedor				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID da avaliação	int8	SIM	PK
rating	Nota da avaliação	numeric	SIM	
comment	Comentário da avaliação (caso exista)	text	NÃO	
created_at	Data de criação da avaliação	timestamptz	SIM	
reviewer_id	ID do avaliador	int4	SIM	FK
seller_id	ID do vendedor sendo avaliado	int4	SIM	FK

Tabela de Avaliações de Vendedor			
edited	A avaliação foi editada?	bool	SIM

Fonte: Autor (2026).

Quadro 29 – Avaliações de produto

Tabela de Avaliações de Produto				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID da avaliação	int8	SIM	PK
rating	Nota da avaliação	numeric	SIM	
comment	Comentário da avaliação (caso exista)	text	NÃO	
created_at	Data de criação da avaliação	timestamptz	SIM	
product_id	ID do produto sendo avaliado	int8	SIM	FK
reviewer_id	ID do avaliador	int4	SIM	FK
edited	A avaliação foi editada?	bool	SIM	

Fonte: Autor (2026).

Quadro 30 – Imagens de produto

Tabela de Imagens de Produto				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID da imagem	int8	SIM	PK
image	Dados da imagem	varchar	SIM	
product_id	ID do produto para o qual a imagem corresponde	int8	SIM	FK

Fonte: Autor (2026).

Quadro 31 – Variações de produto

Tabela de Variações de Produto				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID da variação	int8	SIM	PK
name	Nome da variação	varchar	SIM	
price	Preço da variação	numeric	SIM	
quantity	Quantidade da variação no estoque	int4	SIM	
product_id	ID do produto para o qual a variação corresponde	int8	SIM	FK

Fonte: Autor (2026).

Quadro 32 – Pedidos

Tabela de Pedidos				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID do pedido	int8	SIM	PK
quantity	Quantidade de itens no pedido	int4	SIM	
total_price	Preço total do pedido	numeric	SIM	
status	Status do pedido	varchar	SIM	
tracking_ code	Código de rastreio do pedido (caso seja enviado através de uma transportadora)	varchar	NÃO	
created_at	Data de criação do pedido	timestamptz	SIM	
updated_at	Data da última atualização do pedido (caso houve)	timestamptz	NÃO	
buyer_id	ID do comprador	int4	SIM	FK
product_id	ID do produto sendo comprado	int8	SIM	FK
variant_id	ID da variação específica do produto sendo comprada	int8	SIM	FK

Tabela de Pedidos			
pickup	O pedido será retirado em mãos?	bool	SIM

Fonte: Autor (2026).

Quadro 33 – Estoque

Tabela de Estoque				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID do estoque	int8	SIM	PK
quantity	Quantidade total do produto disponível no estoque	int4	SIM	
product_id	ID do produto ao qual o estoque corresponde	int8	SIM	FK

Fonte: Autor (2026).

Quadro 34 – Itens do carrinho

Tabela de Itens do Carrinho				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID do item	int8	SIM	PK
quantity	Quantidade do item sendo comprada	int4	SIM	
cart_id	ID do carrinho ao qual o item corresponde	int8	SIM	FK
product_id	ID do produto ao qual o item corresponde	int8	SIM	FK
variant_id	ID da variação específica de um produto que está sendo comprada	int8	SIM	FK

Fonte: Autor (2026).

Quadro 35 – Configurações da plataforma

Tabela de Configurações da Plataforma				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID da configuração	int8	SIM	PK
commission_ rate	Taxa de comissão definida pelo gestor	numeric	SIM	

Fonte: Autor (2026).

Quadro 36 – Comissões

Tabela de Comissões				
Nome	**Conteúdo**	**Tipo**	**Obrigatoriedade**	**Chave?**
id	ID da comissão	int8	SIM	PK
rate	Taxa da comissão definida pelo gestor	numeric	SIM	
gross_ amount	Lucro bruto do vendedor (antes da comissão ser aplicada)	numeric	SIM	
commission_ amount	Quantia em dinheiro que equivale à comissão obtida (lucro do gestor)	numeric	SIM	
net_amount	Lucro líquido do vendedor (após a comissão ser aplicada)	numeric	SIM	
created_at	Data na qual a transação ocorreu	timestamptz	SIM	
order_id	ID do pedido ao qual a transação corresponde	int8	SIM	FK

Fonte: Autor (2026).

3.6 Interfaces

Esta subseção mostra todas as interfaces do sistema, conforme observadas na prática.

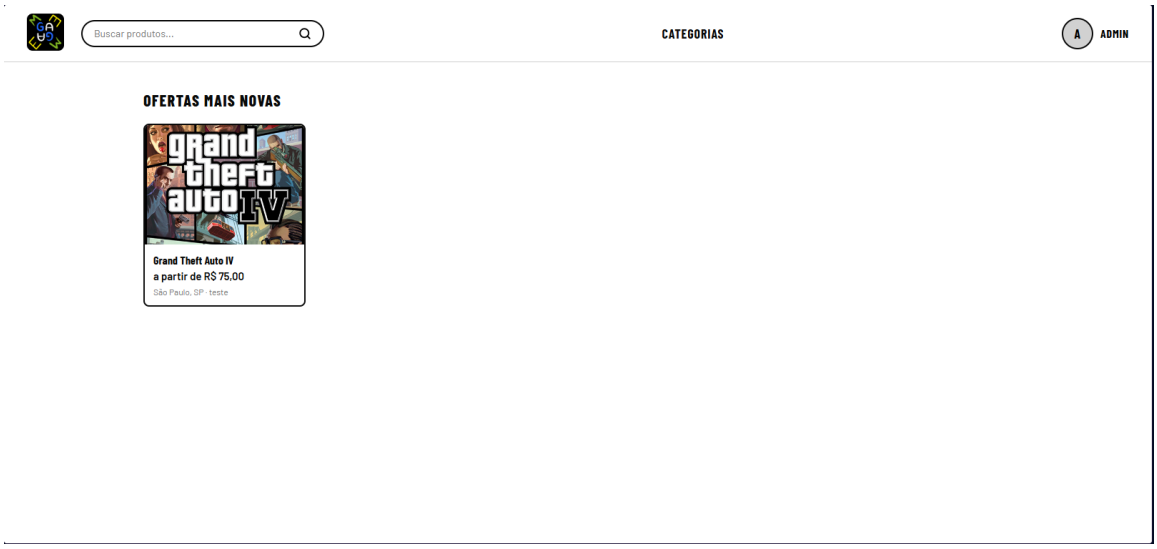


Figura 14 – Captura de tela da *home* do MegaGame

Fonte: Autor (2026).



Figura 15 – Captura de tela da pesquisa do MegaGame

Fonte: Autor (2026).

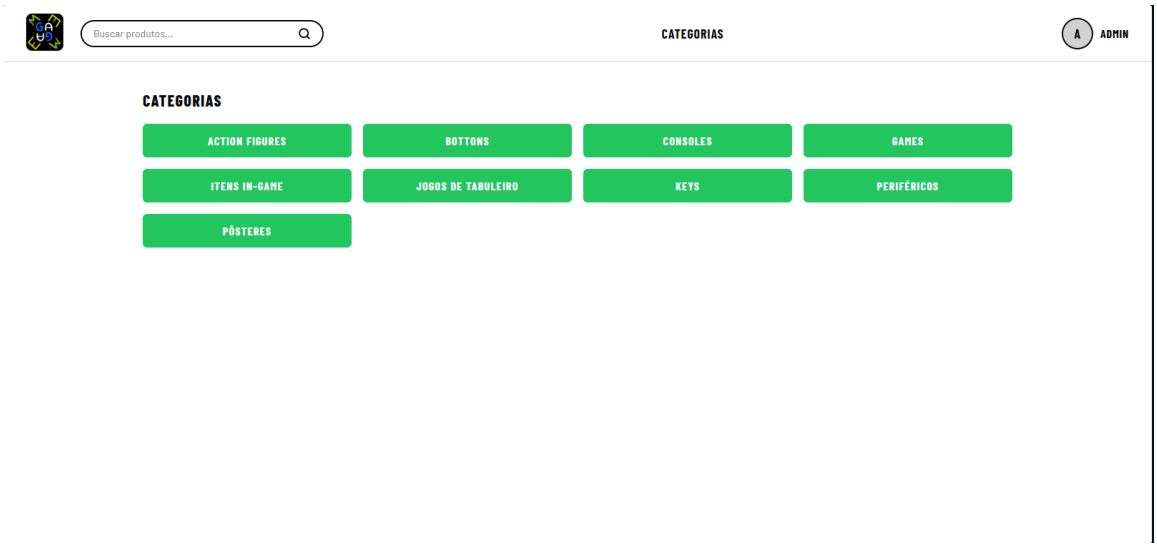


Figura 16 – Captura de tela das categorias do MegaGame

Fonte: Autor (2026).

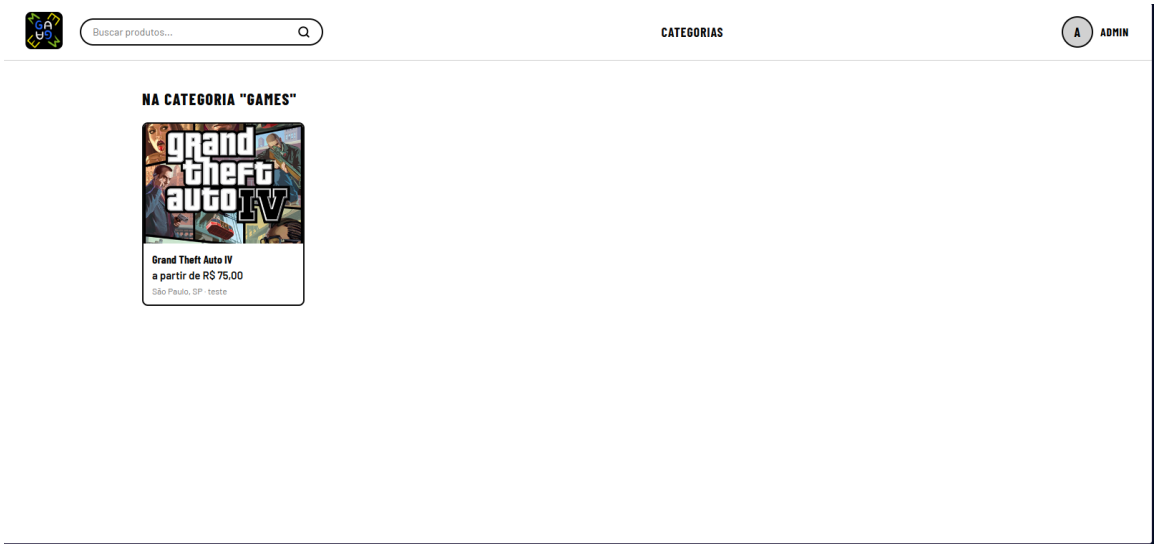


Figura 17 – Captura de tela da busca por categoria do MegaGame

Fonte: Autor (2026).

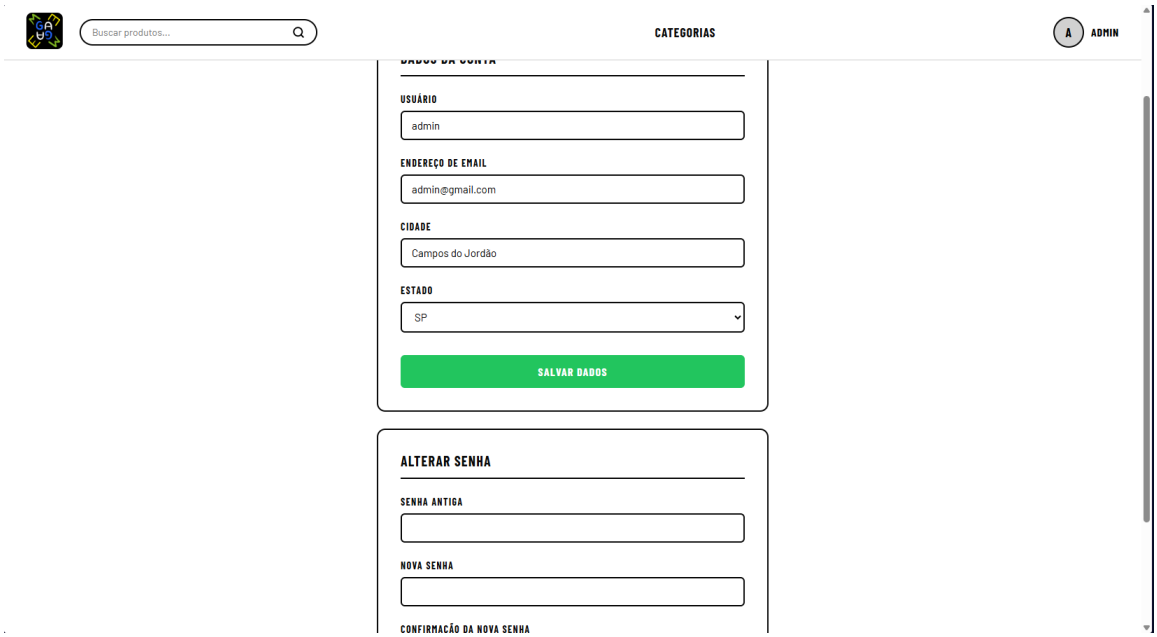


Figura 18 – Captura de tela da edição de perfil do MegaGame

Fonte: Autor (2026).

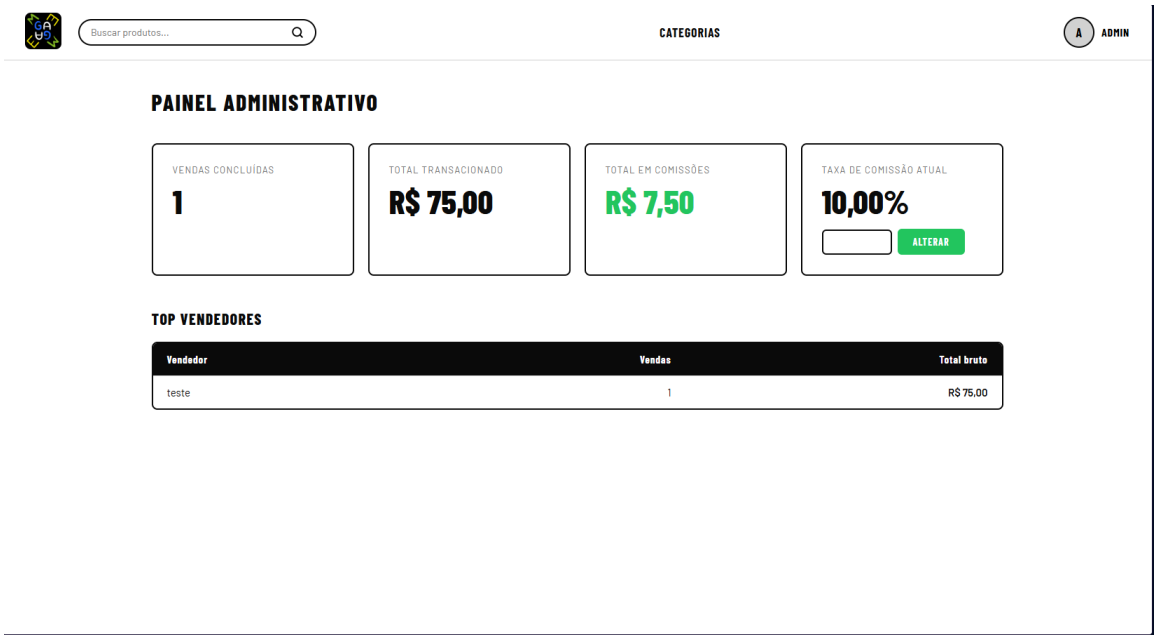


Figura 19 – *Dashboard* de gestor do MegaGame

Fonte: Autor (2026).

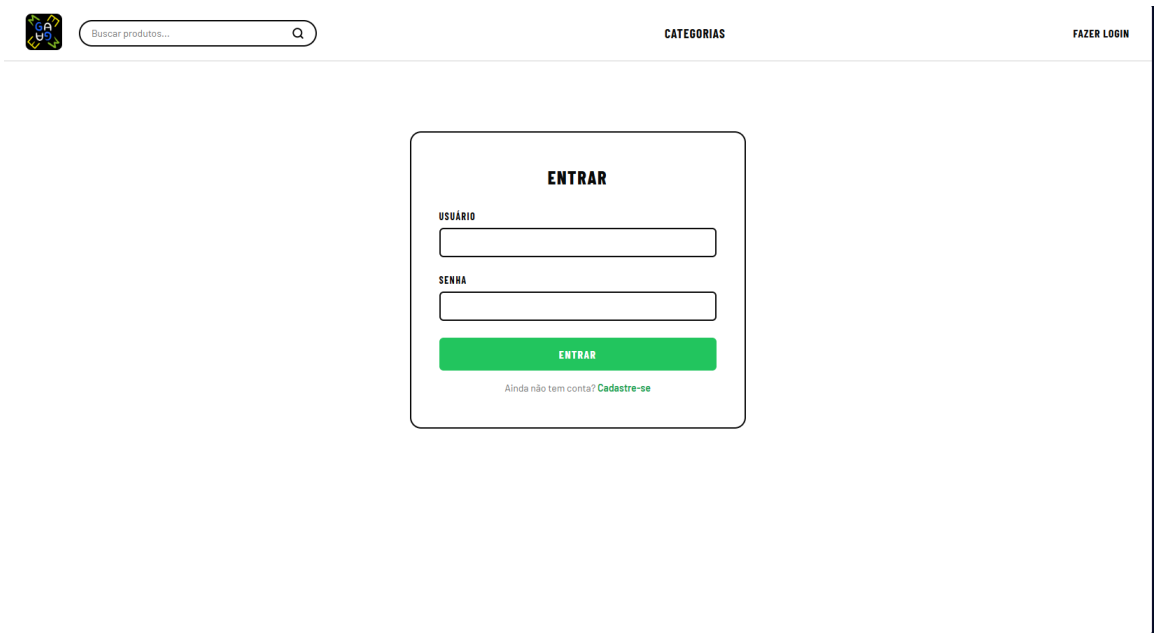


Figura 20 – Captura de tela do *login* do MegaGame

Fonte: Autor (2026).

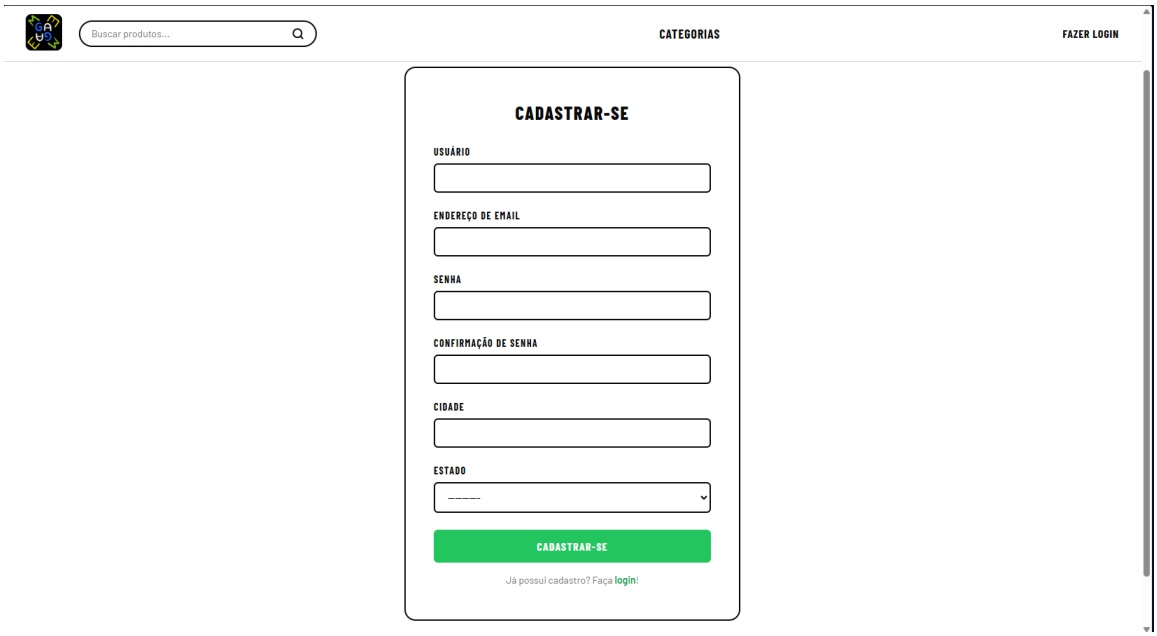


Figura 21 – Captura de tela do cadastro do MegaGame

Fonte: Autor (2026).

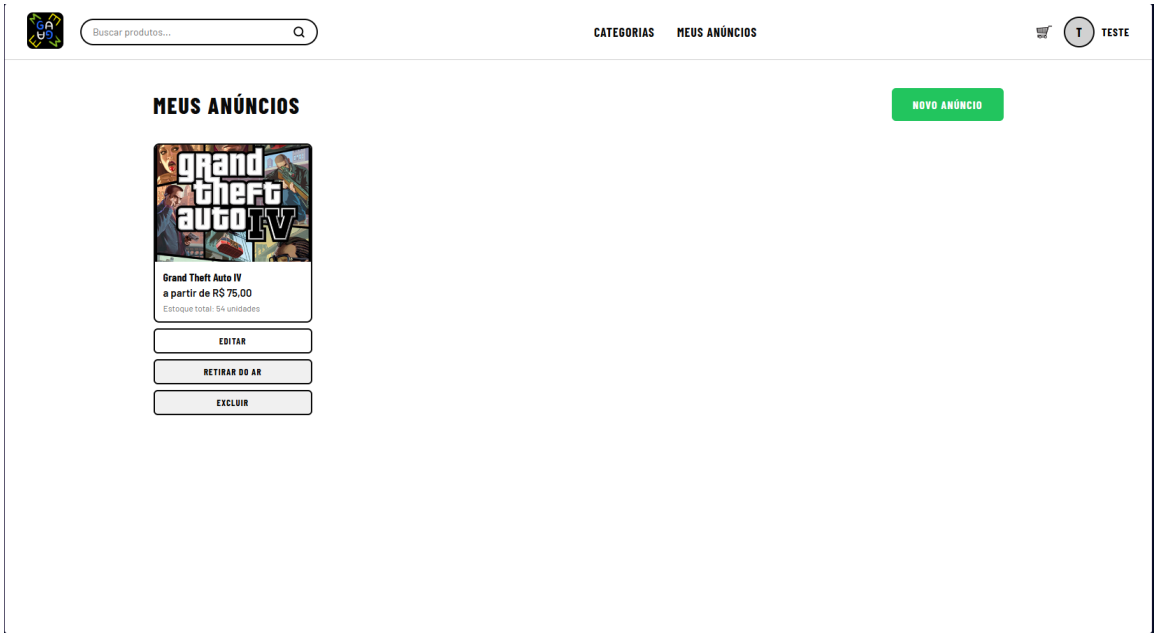


Figura 22 – Captura de tela da lista de anúncios do MegaGame

Fonte: Autor (2026).



Figura 23 – Captura de tela do *dropdown menu* do MegaGame

Fonte: Autor (2026).

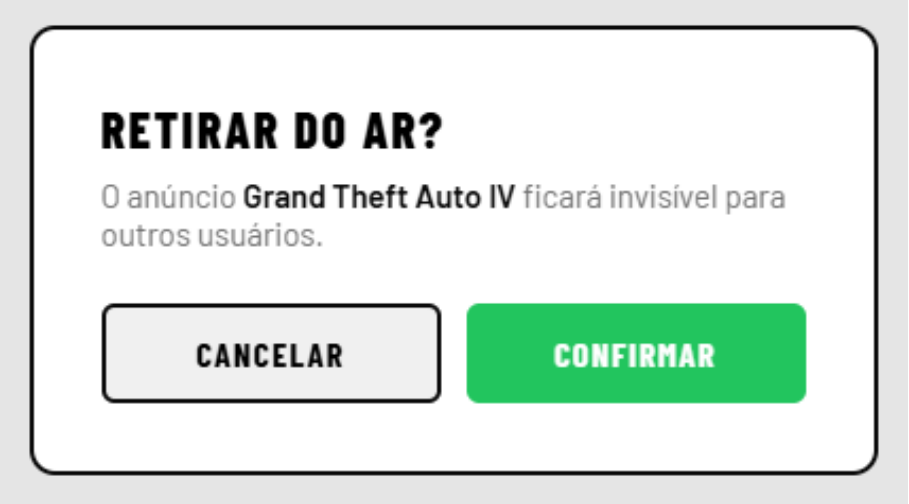


Figura 24 – Captura de tela de um *pop-up* de aviso do MegaGame

Fonte: Autor (2026).

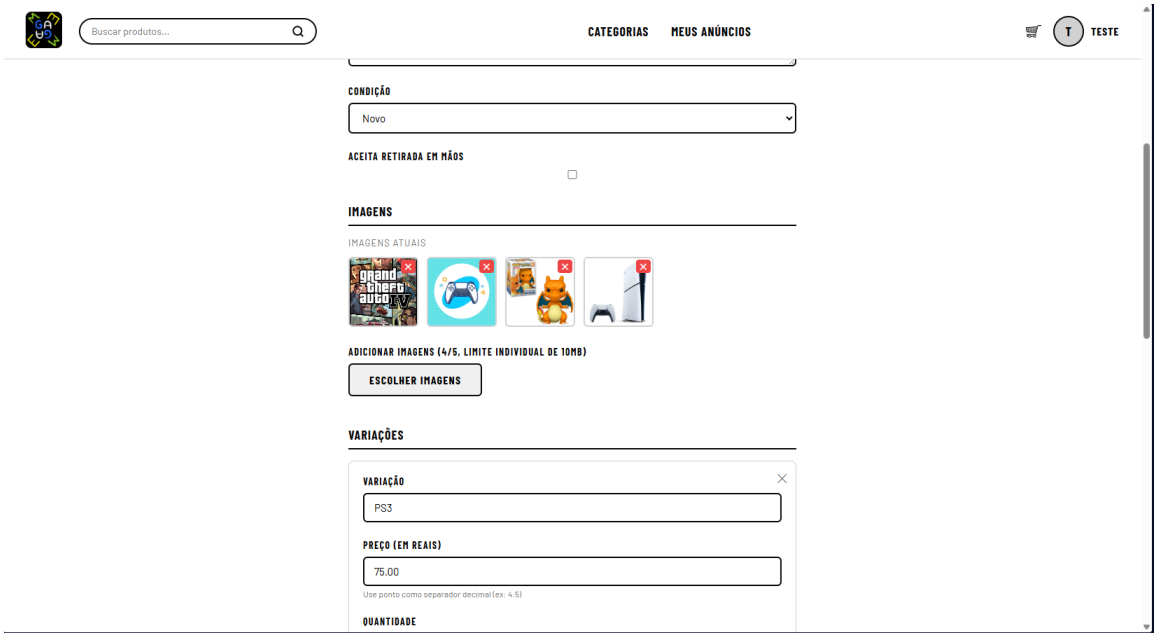


Figura 25 – Captura de tela da edição de anúncio do MegaGame

Fonte: Autor (2026).

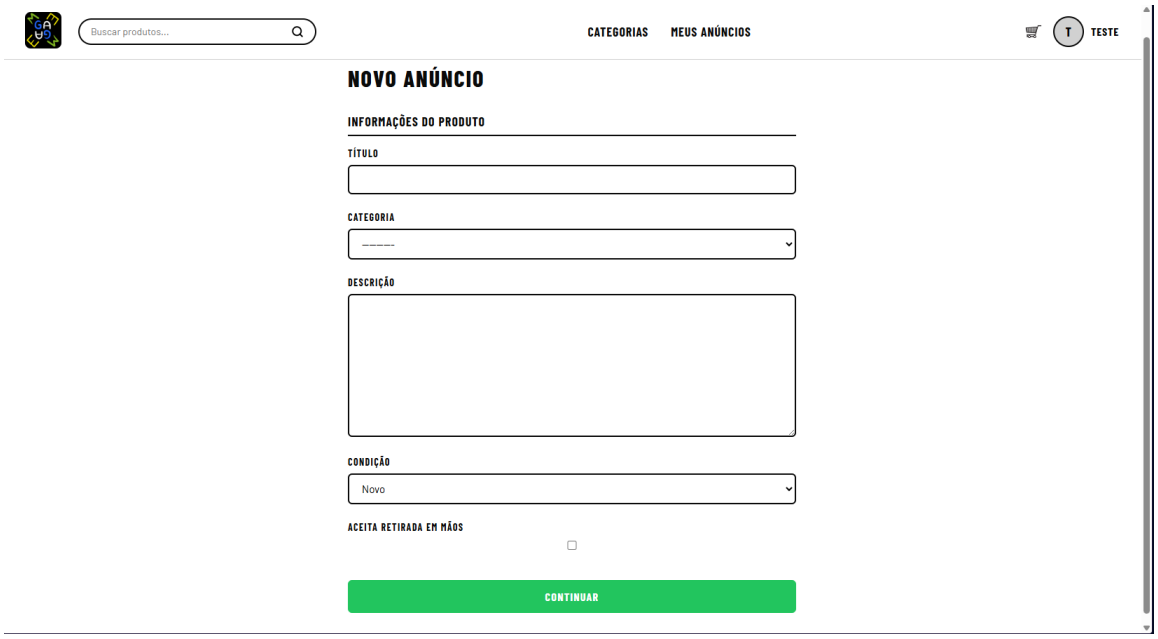


Figura 26 – Captura de tela da criação de anúncio do MegaGame

Fonte: Autor (2026).

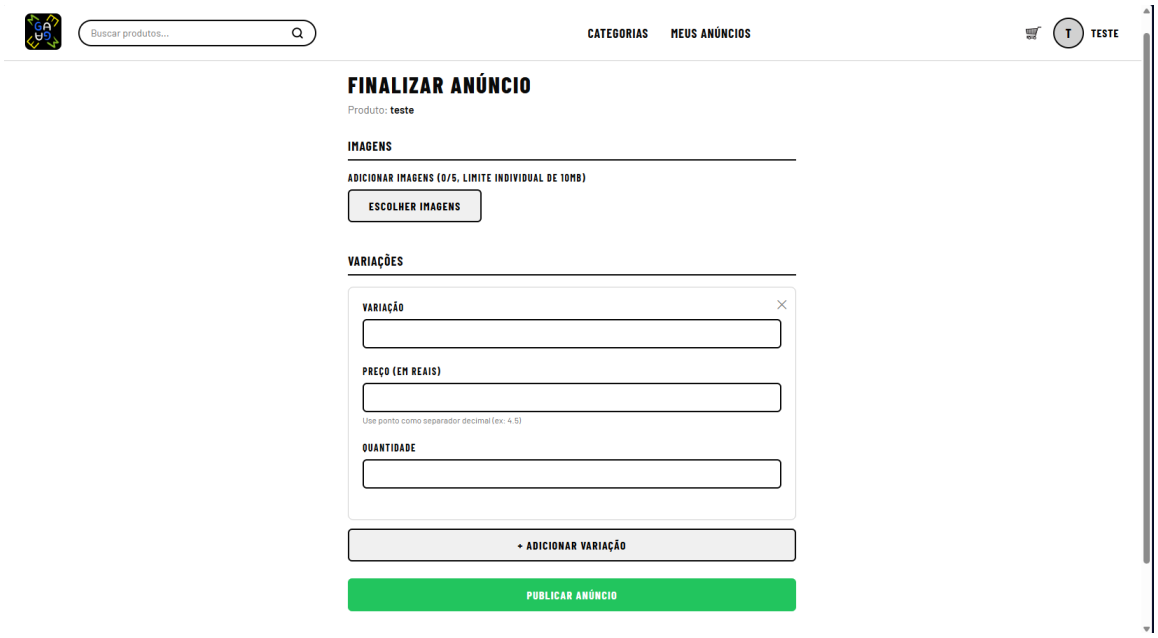


Figura 27 – Criação de variações do MegaGame

Fonte: Autor (2026).



Figura 28 – Rascunhos de anúncio do MegaGame

Fonte: Autor (2026).

3.7 Implementação

O projeto foi construído no Visual Studio Code, IDE da Microsoft. Para rodar o site a partir dele, é necessário primeiramente instalar as dependências do projeto (a partir do arquivo requirements.txt) em um ambiente virtual (venv), configurar core/settings.py e colocar as informações necessárias em um arquivo .env (os possíveis campos estão especificados em .env.example) para que o sistema “aponte” pro banco de dados, executar o comando “python manage.py migrate” no terminal do VS Code para que o banco sendo utilizado receba todas as tabelas necessárias e então executar o comando “python manage.py runserver” quando for necessário inicializar o servidor local.

manage.py é um arquivo na raiz do repositório que, conforme seu nome indica, é responsável por administrar toda a questão da renderização do código (lembrando da prática do SSR empregada aqui) quando “invocado”. Apesar de funcionar apenas

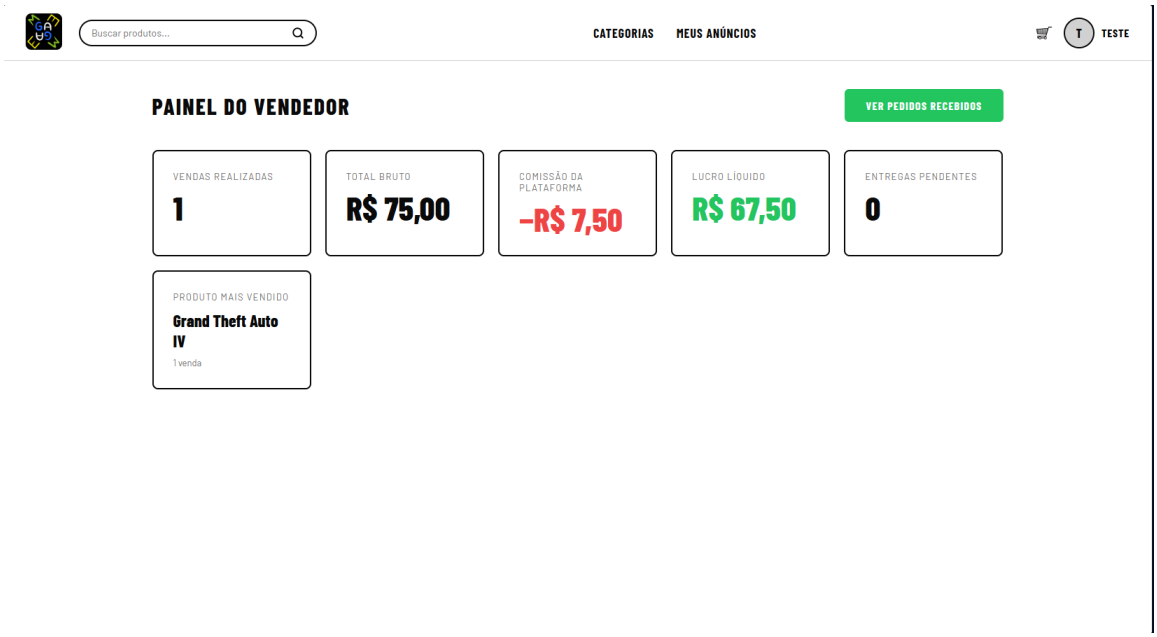


Figura 29 – *Dashboard* de vendedor do MegaGame

Fonte: Autor (2026).

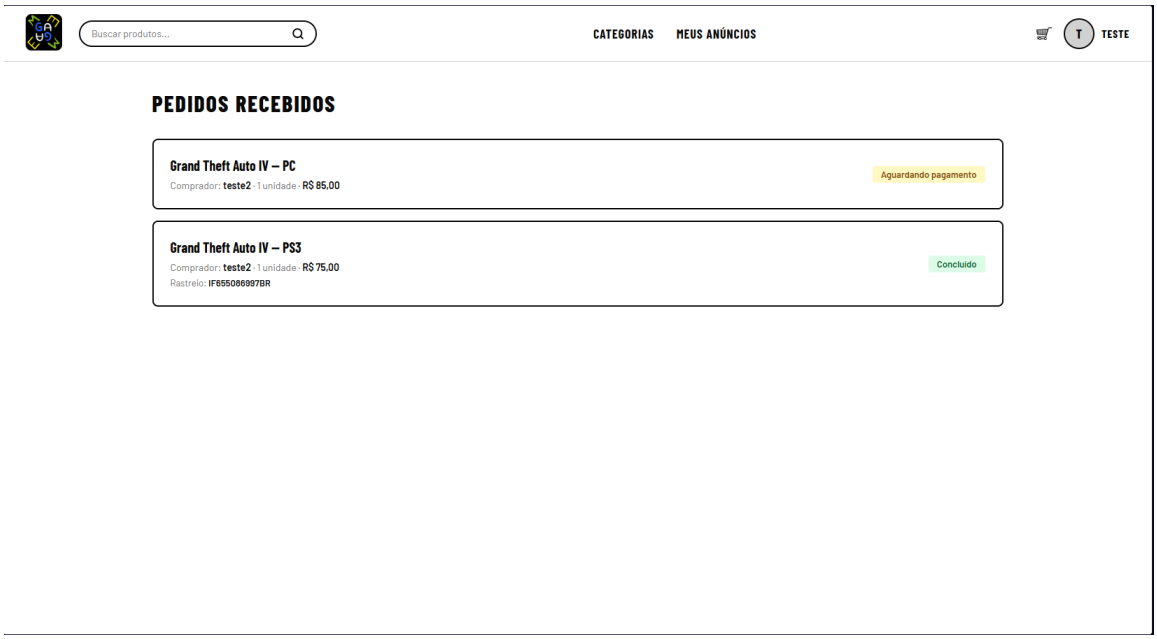


Figura 30 – Captura de tela dos pedidos recebidos do MegaGame

Fonte: Autor (2026).

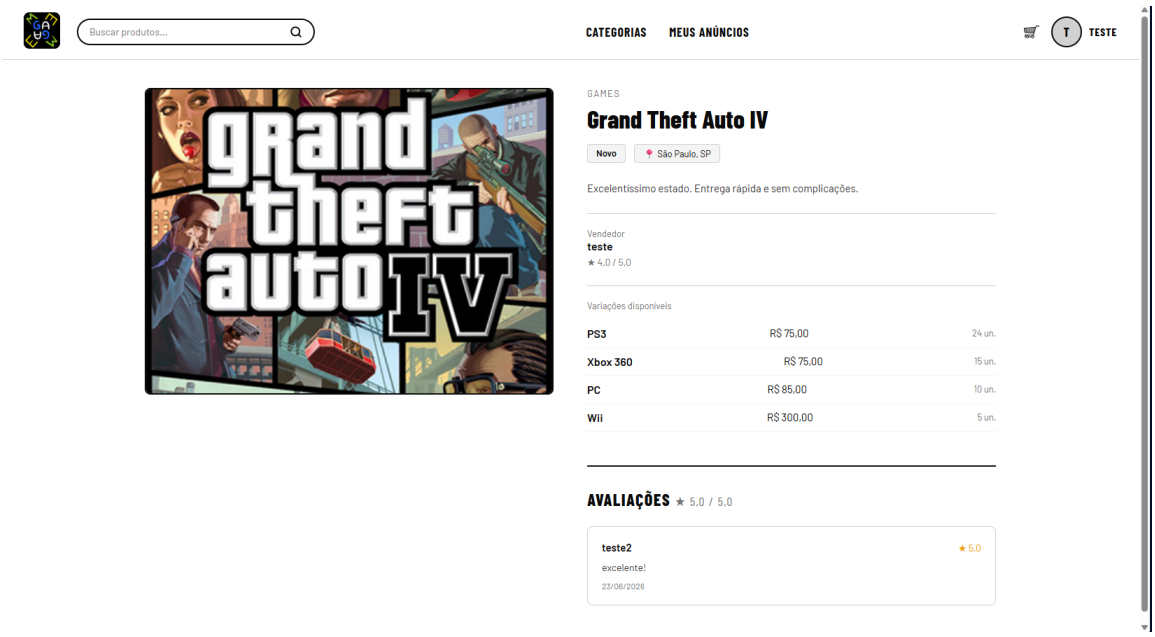


Figura 31 – Captura de tela dos anúncios do MegaGame

Fonte: Autor (2026).

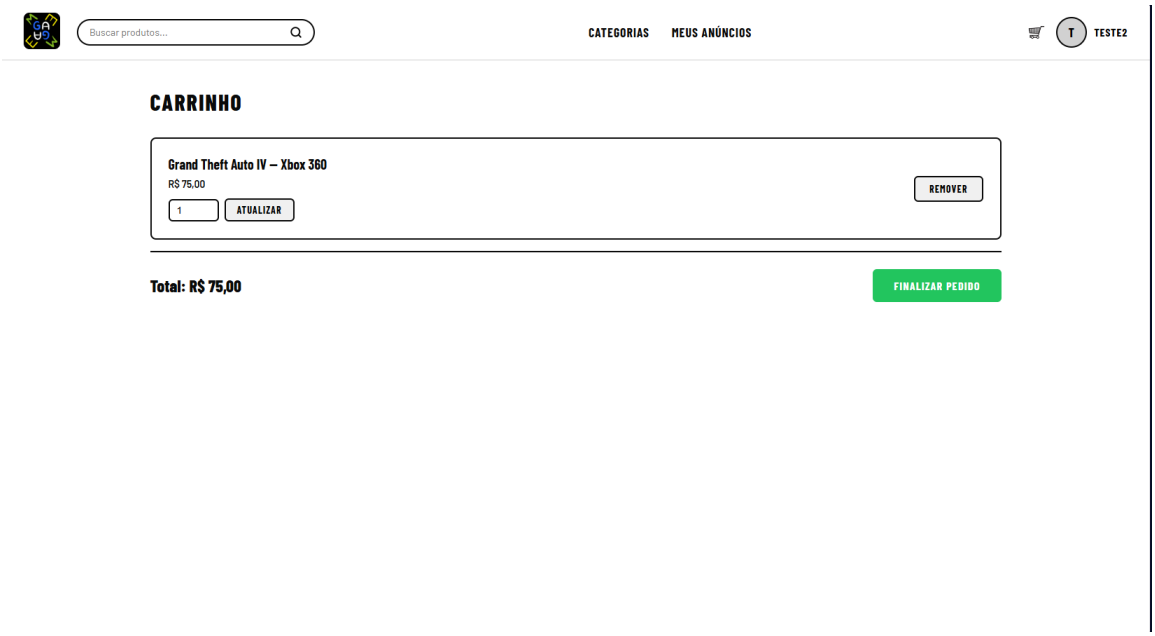


Figura 32 – Captura de tela do carrinho do MegaGame

Fonte: Autor (2026).

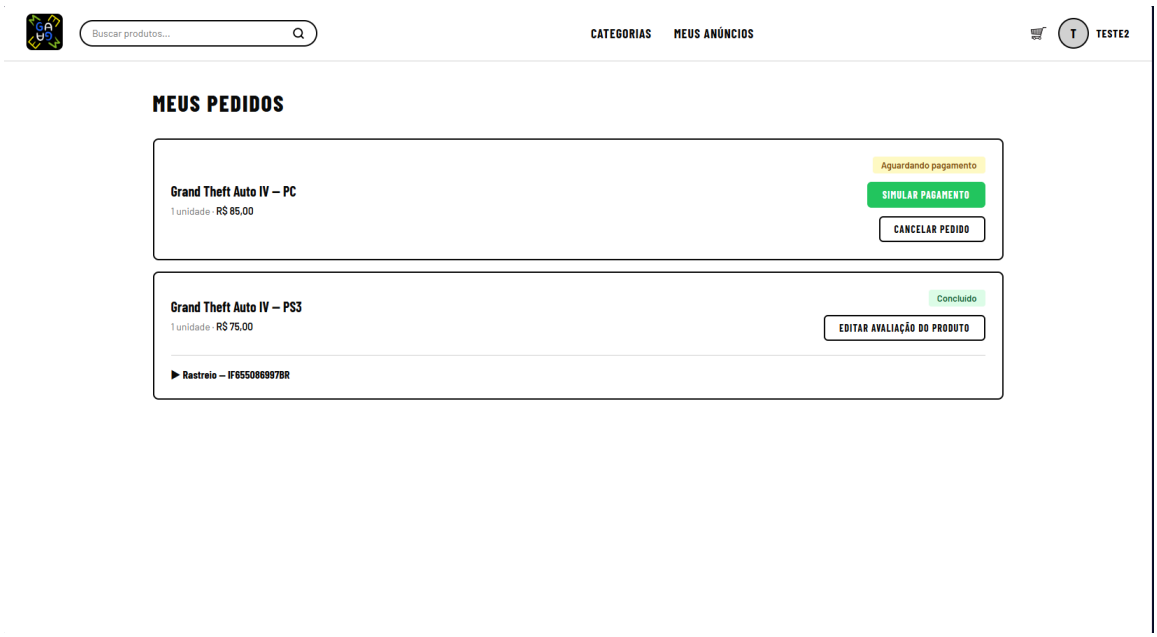


Figura 33 – Captura de tela dos pedidos feitos do MegaGame

Fonte: Autor (2026).

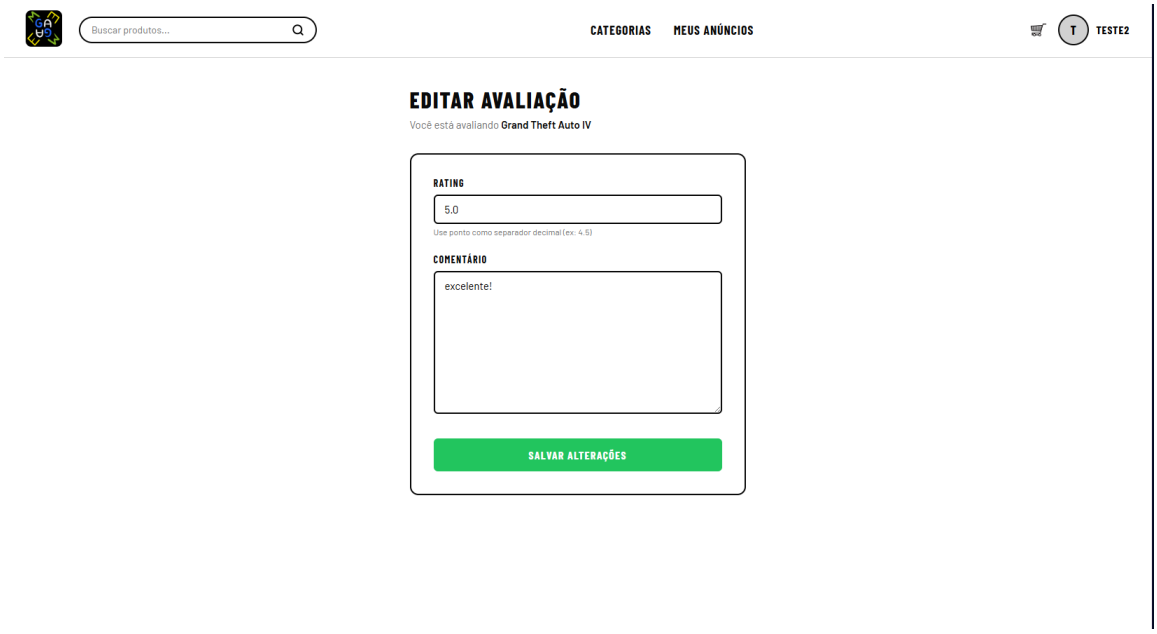


Figura 34 – Avaliação de produto do MegaGame

Fonte: Autor (2026).

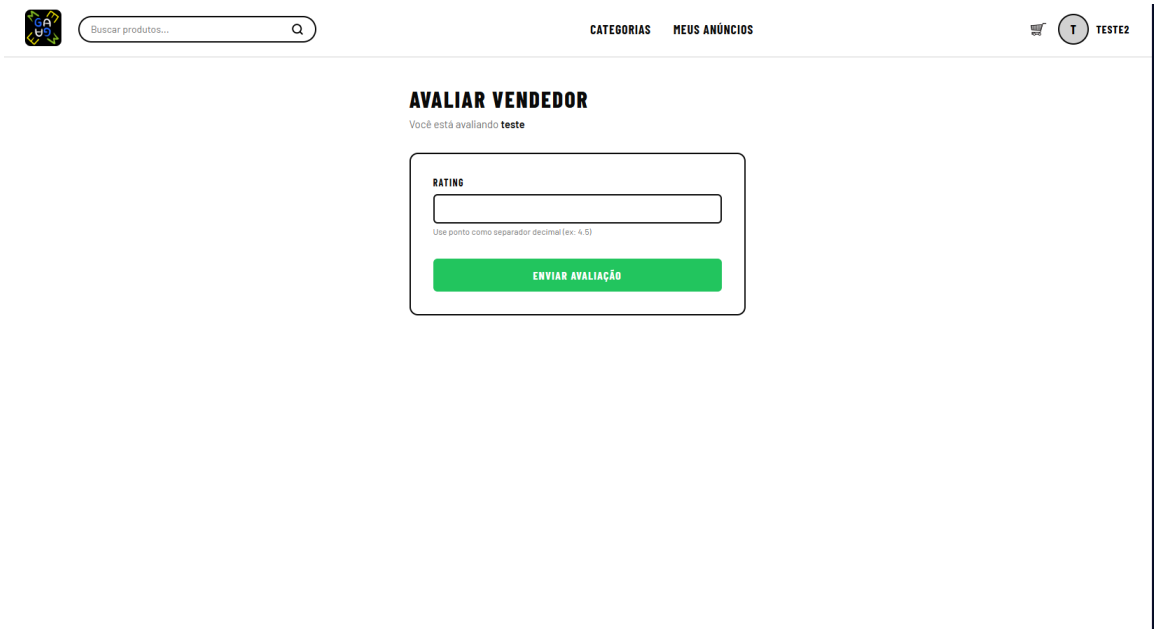


Figura 35 – Avaliação de vendedor do MegaGame

Fonte: Autor (2026).

de forma local no momento, a infraestrutura já foi projetada para estar pronta pra um *deploy* real na *web* conforme o projeto avança.

4 CONCLUSÃO

Retomando as premissas deste trabalho, o objetivo era apresentar o desenvolvimento do MegaGame, um *marketplace* C2C projetado especificamente para preencher uma lacuna identificada no cenário nacional de jogos eletrônicos, que era a falta de uma plataforma de comércio dedicada a esse segmento. Através dos passos detalhados no decorrer do documento, foi possível projetar e implementar um espaço otimizado para o comércio de produtos relacionados a jogos, sendo este funcional, seguro e escalável.

As escolhas metodológicas tomadas para cumprir a meta obtiveram êxito. No *back end*, o Django com SSR garantiu robustez e estabilidade, enquanto o Supabase proporcionou uma infraestrutura de dados confiável e pronta pra *deploy*. Assim, atenderam-se tanto as necessidades operacionais comuns quanto os princípios guiadores da engenharia de *software*.

Adicionalmente, a arquitetura escolhida, tal como o uso do UML para mapear as funcionalidades do sistema, garante que seja possível evoluir a ideia cada vez mais confome o tempo, independente de quem esteja no comando, ao fornecer *insights* claros e objetivos sobre como tudo é operado.

Em suma, o projeto demonstra a viabilidade técnica de uma solução comercial voltada ao público *gamer*, integrando conceitos teóricos de gestão de projetos com ferramentas consolidadas do mercado tecnológico. Com isso, o MegaGame se estabelece como uma resposta prática a uma oportunidade de mercado atual e também como uma aplicação pronta para ser adaptada de acordo com futuras demandas do setor.

REFERÊNCIAS

GUEDES, T. A. G. *UML 2: uma abordagem prática*. 3. ed. São Paulo: Novatec, 2018.

MORRIS, S.; CORONEL, C. *Sistemas de banco de dados: projeto, implementação e administração*. 12. ed. São Paulo: Cengage, 2016.

Newzoo. *Top countries and markets by gaming revenues*. s.d. Disponível em: <https://newzoo.com/resources/rankings/top-10-countries-by-game-revenues>. Acesso em: 2026-06-22.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 9. ed. São Paulo: AMGH, 2021.

SOMMERVILLE, I. *Engenharia de software*. 9. ed. São Paulo: Pearson, 2011.

Supabase. *Supabase Platform*. 2026. Disponível em: <https://supabase.com/docs/guides/platform>. Acesso em: 2026-06-22.

Wikipedia. *Django (web framework)*. 2026. Disponível em: <https://en.wikipedia.org/wiki/Django_(web_framework)>. Acesso em: 2026-06-22.

Wikipedia. *Python (programming language)*. 2026. Disponível em: <https://en.wikipedia.org/wiki/Python_(programming_language)>. Acesso em: 2026-06-22.

Wikipedia. *Unified Modeling Language*. 2026. Disponível em: <https://en.wikipedia.org/wiki/Unified_Modeling_Language>. Acesso em: 2026-06-22.